

UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR DE INGENIEROS DE TELECOMUNICACIÓN



GRADO EN INGENIERÍA DE TECNOLOGÍAS Y SERVICIOS DE
TELECOMUNICACIÓN

TRABAJO FIN DE GRADO

DISEÑO E IMPLEMENTACIÓN DE
VOCTOMIX 2.0

MARTÍN HERRANZ SÁNCHEZ

2 de febrero de 2026

GRADO EN INGENIERÍA DE TECNOLOGÍAS Y SERVICIOS DE TELECOMUNICACIÓN

TRABAJO FIN DE GRADO

Título: Diseño e implementación de Voctomix 2.0

Autor: D. Martín Herranz Sánchez

Tutor: D. ?????

Ponente: ?????

Departamento: ?????

MIEMBROS DEL TRIBUNAL

Presidente: D. ?????

Vocal: D. ?????

Secretario: D. ?????

Suplente: D. ?????

Los miembros del tribunal arriba nombrados acuerdan otorgar la calificación de:
?????

Madrid, a de de 2026

UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR DE INGENIEROS DE TELECOMUNICACIÓN



GRADO EN INGENIERÍA DE TECNOLOGÍAS Y SERVICIOS DE
TELECOMUNICACIÓN

TRABAJO FIN DE GRADO

Diseño e implementación de Voctomix 2.0

Martín Herranz Sánchez

2 de febrero de 2026

Resumen

La producción audiovisual en directo ha evolucionado desde grandes estudios con equipamiento dedicado de alto coste hacia soluciones basadas en software que permiten desplegar un sistema profesional de realización con hardware convencional. Voctomix es un mezclador de vídeo de código abierto desarrollado sobre GStreamer y Python que posibilita esta democratización de la producción en directo.

Este Trabajo Fin de Grado presenta el diseño e implementación de Voctomix 2.0, una versión extendida del mezclador original con nuevas capacidades orientadas a la producción remota. Las contribuciones principales del proyecto son: la integración de un sistema de gestión de composites avanzados (PIP, side-by-side, fullscreen), un módulo de transiciones basadas en mezcla de señales, soporte para overlays gráficos con animaciones de entrada y salida, gestión dinámica del audio mediante un servicio dedicado, y un sistema de telemetría basado en RabbitMQ que registra en tiempo real todos los eventos de sesión.

Para facilitar el despliegue reproducible del sistema completo, se ha desarrollado una imagen Docker junto con un fichero Docker Compose que orquesta voctocore, las fuentes de prueba FFmpeg, el broker RabbitMQ y el servicio de telemetría en una red privada aislada. Este despliegue permite reproducir el entorno completo en cualquier máquina con un único comando.

El sistema ha sido validado en dos escenarios: un despliegue en dos equipos físicos en red local y un despliegue completo en contenedores Docker. Los resultados confirman una latencia de conmutación inferior a un fotograma (<40 ms) y el registro correcto de todos los eventos de sesión en RabbitMQ.

Palabras clave: producción audiovisual en directo, mezclador de vídeo, GStreamer, Docker, RabbitMQ, telemetría, software libre, streaming, AMQP.

Summary

Live audiovisual production has evolved from large studios relying on dedicated high-cost hardware towards software-based solutions that allow deploying a professional production system on commodity hardware. Voctomix is an open-source video mixer built on GStreamer and Python that enables this democratisation of live production.

This Bachelor's Thesis presents the design and implementation of Voctomix 2.0, an extended version of the original mixer featuring new capabilities aimed at remote production. The main contributions of the project are: integration of an advanced composite management system (PIP, side-by-side, fullscreen), a signal-mixing-based transitions module, support for graphical overlays with blend-in and blend-out animations, dynamic audio management through a dedicated service, and a RabbitMQ-based telemetry system that records all session events in real time.

To facilitate reproducible deployment of the full system, a Docker image and a Docker Compose file have been developed, orchestrating voctocore, FFmpeg test sources, the RabbitMQ broker, and the telemetry service in an isolated private network. This deployment allows reproducing the complete environment on any machine with a single command.

The system has been validated in two scenarios: a two-physical-machine deployment over a local network and a full Docker container deployment. Results confirm a switching latency below one video frame (<40 ms) and correct logging of all session events in RabbitMQ.

Keywords: live audiovisual production, video mixer, GStreamer, Docker, RabbitMQ, telemetry, free software, streaming, AMQP.

Agradecimientos

En primer lugar, quiero agradecer a mi tutor [Nombre del tutor] su orientación, disponibilidad y los consejos que han guiado el desarrollo de este trabajo desde el primer día. Su apoyo ha sido fundamental para mantener el rumbo del proyecto en los momentos de mayor dificultad.

A mis compañeros de la ETSIT, con quienes he compartido estos años de carrera: gracias por los apuntes prestados, los proyectos en común y, sobre todo, por las horas de estudio que han hecho más llevadero el camino.

A mi familia, por su paciencia infinita durante los periodos de mayor carga de trabajo y por creer siempre en mí incluso cuando yo dudaba. Sin vuestro apoyo, este proyecto no habría sido posible.

Por último, a la comunidad de software libre y, en especial, a los desarrolladores y colaboradores de Voctomix y GStreamer, cuyo trabajo desinteresado es la base sobre la que se sustenta este TFG.

Índice

Resumen	II
Summary	III
Agradecimientos	IV
Lista de acrónimos	x
1. Introducción y objetivos	1
1.1. Introducción	1
1.2. Objetivos	1
1.3. Estructura de la memoria	2
2. Estado del arte	3
2.1. Producción audiovisual	3
2.1.1. Producción audiovisual tradicional	3
2.1.2. Producción audiovisual remota	4
2.2. Contribución Audiovisual	6
2.2.1. Interfaces y protocolos	6
2.2.2. Códecs de contribución	7
2.2.3. Formatos	7
2.3. Distribución Audiovisual	8
2.3.1. Interfaces y protocolos de distribución	8
2.3.2. Códecs de Vídeo	9
2.3.3. Formatos	10
3. Diseño e implementación del sistema de producción remota de vídeo	11
3.1. Arquitectura del sistema	13
3.1.1. voctocore: núcleo multimedia	13
3.1.2. voctogui: interfaz gráfica de control	13
3.2. Fuentes de señal de cámaras	14
3.2.1. Conexión de las fuentes	14
3.2.2. Previsualización en la interfaz gráfica	14
3.2.3. Requisitos de formato de la señal de entrada	15
3.3. Fuentes de señal auxiliares	15
3.3.1. Fuente break e intro	15
3.3.2. Entrada de audio	16
3.3.3. Stream Blanker: estados de emisión	16
3.4. Composites	17
3.4.1. Full Screen (fs)	17
3.4.2. Picture in Picture (pip)	17
3.4.3. Side by Side (sbs)	17
3.4.4. Lecture (lec)	18

3.4.5.	Mirror	18
3.5.	Transiciones	18
3.5.1.	Cut (corte instantáneo)	18
3.5.2.	Trans (transición suave)	18
3.5.3.	Retake	19
3.6.	Overlays, logos y texto dinámico	19
3.6.1.	Logos	19
3.6.2.	Overlays PNG y texto dinámico	19
3.6.3.	Rotulación dinámica de ponente	20
3.6.4.	Sistema de doble rotulación	20
3.6.5.	Botón UPDATE: recarga dinámica de recursos	20
3.6.6.	Auto-off de overlays	21
3.7.	Señal de programa final	21
3.8.	Personalización de la interfaz gráfica	21
3.8.1.	Tema oscuro y hoja de estilos	22
3.8.2.	Iconografía SVG	22
3.8.3.	Reloj de estudio	22
3.9.	Sistema de telemetría	22
3.9.1.	Arquitectura del servicio de telemetría	23
3.9.2.	API REST y mensajería AMQP	25
3.9.3.	RabbitMQ como broker de mensajería	25
3.9.4.	Evolución del protocolo de telemetría: de UDP a HTTP	25
3.10.	Contenerización Docker	26
3.10.1.	Dockerfile	26
3.10.2.	Docker Compose y patrón <i>Shared Network Namespace</i>	27
3.10.3.	Resiliencia ante condiciones de carrera	27
3.10.4.	Aislamiento híbrido: GUI nativa, servicios en contenedor	27
3.10.5.	Inyección de recursos mediante volúmenes	28
3.10.6.	Patrón <i>Clean Exit</i> : liberación atómica de recursos	28
3.10.7.	Scripts de arranque	28
3.11.	Despliegue en Kubernetes	29
3.11.1.	Estructura de manifiestos	29
3.11.2.	Sondas de preparación y orden de arranque	30
3.11.3.	Script de lanzamiento y port-forward	30
4.	Pruebas de validación del sistema de producción remota de vídeo	31
4.1.	Escenarios de despliegue evaluados	31
4.1.1.	Escenario 1: un PC (entorno de desarrollo)	31
4.1.2.	Escenario 2: dos PCs en red local	31
4.1.3.	Escenario 3: Docker Compose	32
4.1.4.	Escenario 4: Kubernetes con Minikube	32
4.2.	Pruebas funcionales	32
4.2.1.	Conmutación de fuentes y composites	33
4.2.2.	Stream blanker	33
4.2.3.	Overlays y rotulación dinámica	33
4.2.4.	Sistema de telemetría	34
4.2.5.	Validación del despliegue en Docker y Kubernetes	34
4.3.	Pruebas de rendimiento	35
4.3.1.	Metodología	35

4.3.2. Consumo de CPU y memoria	35
4.3.3. Ancho de banda interno	36
4.3.4. Latencia de conmutación	36
4.4. Resumen de resultados	37
5. Conclusiones y líneas futuras	38
5.1. Conclusiones	38
5.2. Líneas de trabajo futuro	39
Bibliografía	40
Anexo A: Aspectos éticos, económicos, sociales y ambientales	43
Anexo B: Presupuesto económico	45

Índice de figuras

2.1. Cadena de producción audiovisual en directo: señales de contribución, mezclador de vídeo y salida de distribución.	5
3.1. Arquitectura del sistema Voctomix 2.0	12
3.2. Disposición visual de las fuentes A y B en los cuatro composites principales de Voctomix 2.0: pantalla completa (fs), imagen en imagen (pip), lado a lado (sbs) y conferencia (lec).	17
3.3. Interfaz gráfica voctogui personalizada	22
3.4. Arquitectura de pods en el despliegue Kubernetes: el pod studio agrupa todos los contenedores del pipeline en un namespace de red compartido; rabbitmq corre como pod independiente; el PVC persiste los registros de sesión.	30
4.1. Consola de administración RabbitMQ mostrando la cola voctomix_events con los eventos registrados durante la sesión de prueba (captura pendiente de sustituir).	34
4.2. Salida de kubect1 get pods con todos los contenedores del sistema en estado Running y Ready (captura pendiente de sustituir).	35
4.3. Consumo medio de CPU y RAM de voctocore por escenario de despliegue, con cuatro fuentes 1080p25 activas. Valores orientativos obtenidos del registro de telemetría (.json1).	35
4.4. Distribución de latencias de conmutación medidas en el escenario de dos PCs (WiFi 5 GHz), $n = 20$ mediciones. El 100 % de las mediciones resultó inferior a un fotograma a 25 fps (40 ms).	36

Índice de tablas

2.1. Comparativa entre producción audiovisual tradicional y producción remota (REMI).	5
2.2. Comparativa de interfaces y protocolos de contribución audiovisual.	7
2.3. Comparativa de protocolos de distribución audiovisual en directo.	9
2.4. Comparativa de códecs de vídeo para distribución en directo.	10
3.1. Puertos del sistema Voctomix 2.0.	13
3.2. Servicios definidos en docker-compose.yml	27
4.1. Comparativa de los cuatro escenarios de despliegue evaluados.	31
4.2. Especificaciones del equipo empleado en las pruebas.	33
4.3. Resultados de las pruebas de conmutación.	33
4.4. Ancho de banda interno estimado según número de fuentes activas.	36

4.5. Resumen de resultados de las pruebas de validación por escenario.	37
5.1. Presupuesto económico	45

Lista de acrónimos

ABR	Adaptive Bitrate (tasa de bits adaptativa)
AMQP	Advanced Message Queuing Protocol
API	Application Programming Interface
AVC	Advanced Video Coding (véase H.264/MPEG-4 AVC)
CDN	Content Delivery Network (red de distribución de contenido)
CPU	Central Processing Unit
DASH	Dynamic Adaptive Streaming over HTTP
FLOSS	Free/Libre and Open Source Software
fps	Frames Per Second (fotogramas por segundo)
GPU	Graphics Processing Unit
GUI	Graphical User Interface (interfaz gráfica de usuario)
HEVC	High Efficiency Video Coding (véase H.265)
HLS	HTTP Live Streaming
HTTP	HyperText Transfer Protocol
ITU-T	International Telecommunication Union – Telecommunication Standardization Sector
JSON	JavaScript Object Notation
MPEG	Moving Picture Experts Group
NDI	Network Device Interface
OBS	Open Broadcaster Software
PIP	Picture-in-Picture
REST	Representational State Transfer
RTMP	Real-Time Messaging Protocol
RTSP	Real-Time Streaming Protocol
SDI	Serial Digital Interface
SMPTE	Society of Motion Picture and Television Engineers
SRT	Secure Reliable Transport
TCP	Transmission Control Protocol
TFG	Trabajo Fin de Grado
UDP	User Datagram Protocol
VBR	Variable Bitrate (tasa de bits variable)
VP9	Video codec desarrollado por Google (estándar abierto)
YAML	YAML Ain't Markup Language

1. Introducción y objetivos

1.1. Introducción

La producción audiovisual en directo ha experimentado en los últimos años una profunda transformación, impulsada por la democratización del hardware de bajo coste y el crecimiento exponencial del *streaming* en Internet. Eventos técnicos de referencia como el Chaos Communication Congress (CCC), organizado por el Chaos Computer Club en Alemania, han puesto de manifiesto la necesidad de disponer de herramientas de realización profesional que sean al mismo tiempo económicas, fiables y auditables.

En este contexto surge Voctomix, desarrollado por el equipo C3VOC (*Chaos Communication Congress Video Operations Crew*). Frente a soluciones previas como *dvswitch*, limitada en resolución y escalabilidad, o herramientas comerciales como vMix o OBS Studio, diseñadas para un único operador en un único equipo, Voctomix adopta una arquitectura cliente-servidor desacoplada que separa el motor de mezcla multimedia de la interfaz de control, permitiendo distribuir la carga de procesamiento en distintos nodos de red.

El presente Trabajo de Fin de Grado toma como punto de partida la versión original de Voctomix y propone una serie de extensiones orientadas a su uso en producciones en directo de mediana escala: rotulación dinámica sobre el vídeo, gestión avanzada del blanking de stream, composites avanzados de vídeo, telemetría en tiempo real mediante un broker de mensajería, y la contenerización completa del sistema mediante Docker y Docker Compose. El resultado es Voctomix 2.0, una plataforma más versátil, desplegable y adaptada a flujos de trabajo profesionales.

Un elemento diferenciador de este proyecto es su orientación hacia un **caso de uso real**: el sistema desarrollado está previsto para su utilización por el grupo de investigación **CyberNemo** de la Universidad Politécnica de Madrid (UPM). Este grupo necesita una solución de producción audiovisual remota que permita retransmitir sus actividades, seminarios y presentaciones con calidad profesional y sin depender de infraestructura hardware costosa. Voctomix 2.0 responde directamente a estas necesidades, proporcionando un entorno reproducible y fácilmente desplegable que puede ponerse en funcionamiento con un único comando (`docker compose up`) en cualquier equipo Linux estándar.

1.2. Objetivos

El objetivo principal de este trabajo es extender las capacidades de Voctomix para cubrir las necesidades de una producción audiovisual en directo profesional, manteniendo la filosofía *open-source* y la arquitectura desacoplada del proyecto original.

Para alcanzar este objetivo general se definen los siguientes objetivos específicos:

1. Implementar un sistema de rotulación dinámica que permita superponer gráficos y textos sobre el vídeo en tiempo real, con fundido de entrada/salida configurable.
2. Desarrollar un mecanismo de *stream blanker* que intercale automáticamente bucles de pausa y fuera de emisión, gestionando de forma independiente el audio y el vídeo de cada estado.
3. Incorporar la funcionalidad *Audio Follows Video*, de modo que al seleccionar una fuente de vídeo el sistema active automáticamente su canal de audio asociado.
4. Implementar un servicio de telemetría (*notario*) que exporte el estado de la sesión en formato JSON y lo publique en un broker de mensajería AMQP, permitiendo su integración con sistemas externos de monitorización.
5. Contenerizar el sistema completo mediante Docker y Docker Compose, garantizando la reproducibilidad del entorno y simplificando el despliegue en producción.
6. Validar el sistema en tres escenarios de despliegue distintos: monopuesto, cliente-servidor en red local, y arquitectura completamente contenerizada.

1.3. Estructura de la memoria

La presente memoria se organiza en cinco capítulos y dos anexos:

El **Capítulo 2** presenta el estado del arte en producción audiovisual: la evolución desde la producción tradicional hasta la producción remota basada en software, las tecnologías de contribución y distribución de vídeo, y una comparativa de los mezcladores de vídeo disponibles tanto en el ámbito hardware como en software.

El **Capítulo 3** describe el diseño e implementación del sistema de producción remota de vídeo Voctomix 2.0: la arquitectura del sistema (voctocore y voctogui), las fuentes de señal de cámaras y auxiliares, los modos de composición, las transiciones, el sistema de overlays y rotulación dinámica, la telemetría mediante RabbitMQ, y el despliegue contenerizado con Docker.

El **Capítulo 4** recoge las pruebas de validación del sistema en dos escenarios de despliegue: el escenario de dos PCs en red local y el escenario Docker. Se presentan mediciones de latencia, capturas de los registros de telemetría en RabbitMQ y evidencias del correcto funcionamiento del sistema.

El **Capítulo 5** presenta las conclusiones del trabajo y las líneas de trabajo futuro identificadas, que incluyen el despliegue en Kubernetes y la validación con cámaras reales en el entorno del grupo CyberNemo de la UPM.

Los **Anexos** incluyen información complementaria sobre los aspectos éticos, económicos, sociales y ambientales del proyecto (Anexo A), así como el presupuesto económico detallado (Anexo B).

2. Estado del arte

En este capítulo se presentan las bases tecnológicas sobre las que se sustenta Voctomix 2.0. Se analizan los tres eslabones de la cadena de producción audiovisual en directo: el modelo de producción y las herramientas de realización disponibles (Sección 2.1), los estándares e interfaces de contribución de señal (Sección 2.2), y los protocolos y códecs empleados en la distribución del contenido al espectador final (Sección 2.3). El objetivo es que el lector comprenda el contexto técnico en el que se enmarca el trabajo antes de abordar el diseño e implementación del sistema en el Capítulo 3.

2.1. Producción audiovisual

La producción audiovisual en directo engloba el conjunto de procesos técnicos y organizativos que permiten capturar, procesar y distribuir contenido de vídeo y audio en tiempo real. Históricamente anclada a infraestructuras físicas costosas y al desplazamiento de equipos al lugar del evento, esta disciplina ha experimentado en la última década una transformación radical impulsada por la convergencia entre las redes de datos y los flujos de media profesionales [1].

2.1.1. Producción audiovisual tradicional

El modelo de producción tradicional se basa en el despliegue físico de toda la infraestructura técnica y del personal en el emplazamiento del evento. El componente central de este modelo es el **mezclador hardware**, denominado *switcher*, un dispositivo dedicado que selecciona en cada instante qué fuente de vídeo se emite a la salida de programa y gestiona las transiciones y los efectos. Fabricantes como Blackmagic Design (ATEM), Ross Video o Grass Valley (Kayenne) dominan el mercado de estos equipos, que ofrecen latencias sub-imagen, múltiples buses de programa y previsualización simultáneos, y redundancia hardware certificada para operación continua [2].

En el modelo tradicional, todas las fuentes, cámaras, reproductores de vídeo y ordenadores de presentación, se interconectan mediante cableado **SDI** (*Serial Digital Interface*), el estándar histórico de la industria broadcast para el transporte de vídeo sin comprimir. La señal resultante recorre el mezclador, los sistemas de rotulación y grabación, y finalmente el encoder de distribución, todo ello dentro del mismo emplazamiento físico.

Este modelo ha demostrado su solidez durante décadas en las grandes producciones de televisión: retransmisiones deportivas, eventos parlamentarios y galas de premios emplean esta arquitectura con decenas de fuentes de cámara gestionadas en tiempo real. Sin embargo, su principal limitación es económica y logística: el coste de adquirir y transportar

el equipamiento, más el personal técnico necesario para su instalación y operación, lo pone fuera del alcance de organizaciones de mediana escala.

2.1.2. Producción audiovisual remota

La producción remota, conocida en la industria por el acrónimo **REMI** (*Remote Integration Model*), surge para resolver estas limitaciones permitiendo que la realización se lleve a cabo desde un centro de producción centralizado, mientras en el lugar del evento solo se mantiene el equipo técnico mínimo necesario para la captura de señal [1].

Casos de uso reales

La adopción del modelo REMI ha crecido exponencialmente en los últimos años. Un ejemplo emblemático fue la producción audiovisual de los Juegos Olímpicos de Tokio 2021 por parte de Olympic Broadcasting Services (OBS), que produjo más de 9.500 horas de contenido con la realización, mezcla de señales y control de calidad realizados desde el International Broadcast Centre y desde centros remotos distribuidos por todo el mundo [3]. En España, Telefónica Servicios Audiovisuales (TSA) ejecutó en 2022 la producción remota multicámara de un partido de la UEFA Champions League disputado en el estadio Santiago Bernabéu, realizando toda la dirección desde las instalaciones de Telefónica en Tres Cantos, a más de 30 km del estadio [4].

Ventajas del modelo remoto

Las ventajas del modelo REMI frente a la producción tradicional son significativas [1]:

- **Reducción de costes:** se minimiza el desplazamiento de personal y equipos. Solo se requiere in situ el equipo de captura; la realización, mezcla y edición se realizan de forma centralizada.
- **Sostenibilidad:** la reducción de desplazamientos contribuye a disminuir la huella de carbono de las producciones, un factor cada vez más valorado por organizaciones y broadcasters.
- **Escalabilidad:** el sistema puede adaptarse a producciones de diferente escala reutilizando la misma infraestructura centralizada.
- **Flexibilidad geográfica:** es posible retransmitir eventos desde cualquier parte del mundo con conectividad IP, sin necesidad de desplazar el equipo de producción.

Herramientas de producción remota por software

La disponibilidad de herramientas de mezclado por software ha democratizado el acceso al modelo remoto incluso para organizaciones sin presupuesto broadcast:

- **OBS Studio:** solución de código abierto ampliamente utilizada para *streaming* y grabación. Su arquitectura monolítica está orientada a un único operador en un único equipo, lo que la hace ideal para producciones individuales pero limita su uso en entornos distribuidos.
- **vMix:** solución comercial profesional para Windows con soporte NDI, multiview y producción 4K. Su coste de licencia y su restricción al sistema operativo Windows la excluyen de entornos *open-source* sobre Linux.

- **Voctomix**: herramienta de código abierto con arquitectura cliente-servidor que separa el motor de mezcla (**voccore**) de la interfaz de control (**vocgui**), permitiendo que el procesamiento se realice en un servidor mientras el operador trabaja desde otro equipo conectado por red. Es la herramienta extendida en el presente trabajo (véase el Capítulo 3).

El motor multimedia sobre el que se construye Voctomix es **GStreamer**, un *framework* de código abierto cuya arquitectura de *pipeline* permite construir cadenas de composición de vídeo arbitrariamente complejas a partir de elementos reutilizables [5]. El elemento **compositor** de GStreamer 1.x actúa como mezclador de vídeo multipista: acepta N fuentes de entrada con transformaciones geométricas independientes y produce un único flujo de salida modificable en tiempo real, posibilitando transiciones suaves sin interrumpir el procesamiento.

Tabla 2.1: Comparativa entre producción audiovisual tradicional y producción remota (REMI).

Parámetro	Tradicional	REMI / Software
Infraestructura en lugar del evento	Completa	Mínima (cámaras + encoder)
Coste de equipamiento	Muy alto	Bajo (hardware convencional)
Personal in situ	Numeroso	Reducido
Latencia extremo a extremo	Sub-imagen	1 fotograma – varios segundos
Escalabilidad	Limitada	Alta
Sostenibilidad (huella de carbono)	Baja	Alta
Ejemplo	Blackmagic ATEM	OBS, vMix, Voctomix

Contribución audiovisual

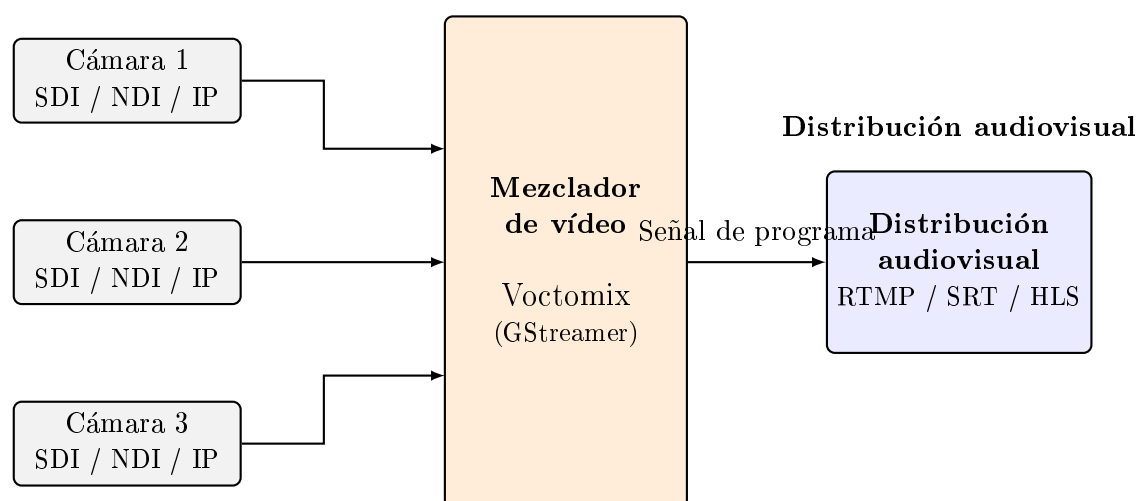


Figura 2.1: Cadena de producción audiovisual en directo: señales de contribución, mezclador de vídeo y salida de distribución.

2.2. Contribución Audiovisual

La contribución audiovisual designa el transporte de la señal de vídeo y audio desde las fuentes de captura, cámaras, ordenadores de presentación y reproductores, hasta el mezclador o núcleo del sistema de producción. Es el primer eslabón de la cadena y su elección determina la calidad máxima disponible, la latencia de extremo a extremo y el coste de la infraestructura. Las opciones disponibles van desde el cableado físico dedicado hasta el transporte sobre redes Ethernet de propósito general.

2.2.1. Interfaces y protocolos

SDI (*Serial Digital Interface*)

SDI es el estándar histórico de la industria broadcast para el transporte de vídeo sin comprimir, definido por la familia de normas SMPTE (259M para SD, 292M para HD, 424M para 3G-SDI y 2081M para 12G-SDI). La señal se transmite sobre cable coaxial o fibra óptica a velocidades que van desde los 270 Mbit/s (SD) hasta los 11,88 Gbit/s (12G-SDI para señales 4K), con latencia inherentemente sub-imagen y sincronismo garantizado entre fuentes [6].

Su principal ventaja es la robustez: la señal SDI no depende de protocolos de red, no sufre variaciones de latencia (*jitter*) y no requiere configuración de red. Su limitación es la distancia (hasta 100 m en coaxial sin regeneradores) y la infraestructura especializada necesaria, cables, distribuidores y conversores, que encarece significativamente las instalaciones.

SMPTE ST 2022-6: SDI sobre IP

SMPTE ST 2022 es una familia de estándares que define el transporte de señales de vídeo broadcast sobre redes IP. Su variante más relevante para contribución, **SMPTE ST 2022-6** (HBRMT, *High Bit Rate Media Transport*), encapsula la señal SDI íntegra, incluyendo audio embebido y datos auxiliares, en paquetes UDP/IP, lo que facilita la migración de infraestructuras SDI existentes hacia redes Ethernet estándar sin necesidad de sustituir el equipamiento de generación de señal [7]. El estándar incorpora mecanismos de corrección de errores FEC para compensar las pérdidas de paquetes propias de las redes IP.

SMPTE ST 2110: Producción IP nativa

SMPTE ST 2110, publicado en 2017, representa la evolución hacia una producción completamente basada en IP. A diferencia de ST 2022-6, que transporta el flujo SDI encapsulado, ST 2110 desacopla los componentes de la señal: vídeo, audio y metadatos se transportan como **flujos IP independientes**, lo que proporciona una flexibilidad de encaminamiento imposible en SDI [8].

La sincronización entre todos los flujos se consigue mediante el protocolo **PTP** (*Precision Time Protocol*, IEEE 1588), que garantiza sincronización con precisión de microsegundos en toda la infraestructura. Adicionalmente, ST 2110 elimina los componentes de la trama SDI innecesarios en entornos IP, logrando ahorros de ancho de banda del **15–30 %** respecto a ST 2022-6 para la misma señal [8]. Su adopción está muy extendida en nuevas instalaciones broadcast profesionales y en estudios de televisión de nueva construcción.

NDI (*Network Device Interface*)

NDI es un protocolo propietario de Vizrt/NewTek que permite enviar vídeo con compresión ligera sobre redes LAN Ethernet estándar con latencia de un fotograma. Es ampliamente soportado por cámaras PTZ, mezcladores hardware modernos y herramientas de software de producción. Su principal ventaja es la facilidad de integración en infraestructuras de red existentes sin necesidad de hardware especializado.

Tabla 2.2: Comparativa de interfaces y protocolos de contribución audiovisual.

Estándar	Medio	Velocidad máx.	Latencia	Uso típico
SDI (12G)	Coaxial/fibra	11,88 Gbit/s	Sub-imagen	Estudio broadcast
SMPTE ST 2022-6	Ethernet IP	3 Gbit/s	Sub-imagen	Migración SDI/IP
SMPTE ST 2110	Ethernet IP	Variable	Sub-imagen	Producción IP nativa
NDI	Ethernet LAN	~100 Mbit/s	1 fotograma	Software, PTZ

2.2.2. Códecs de contribución

En la etapa de contribución se prioriza la calidad y la ausencia de pérdidas sobre la eficiencia de compresión, a diferencia de la distribución al espectador final:

- **Vídeo sin comprimir (RAW):** el vídeo se transporta sin ningún tipo de compresión, preservando toda la información de la señal original. Los formatos más utilizados son UYVY (crominancia 4:2:2 empaquetada, 2 bytes/píxel) e I420 (YUV 4:2:0 planar). Voctomix emplea I420 internamente para todo el procesamiento mediante el motor GStreamer, transportado sobre conexiones TCP en contenedor Matroska.
- **Apple ProRes:** familia de códecs de alta calidad desarrollados por Apple, diseñados para producción profesional. Utilizan exclusivamente compresión intra-frame (cada fotograma es independiente), lo que minimiza la degradación acumulativa en múltiples etapas de procesamiento. ProRes 422 HQ es el estándar en producción televisiva para postproducción y distribución entre equipos de edición [9].
- **Avid DNxHD/DNxHR:** familia equivalente a ProRes desarrollada por Avid, también basada en compresión intra-frame. Ampliamente utilizada en flujos de trabajo de postproducción broadcast.
- **JPEG 2000:** códec que ofrece compresión sin pérdidas (*lossless*) o con pérdidas mínimas y latencia muy reducida gracias a su arquitectura de codificación por *tiles* independientes. Utilizado en sistemas de contribución broadcast de alta calidad y en señalización digital.

2.2.3. Formatos

El formato contenedor define cómo se encapsulan y multiplexan los flujos de vídeo y audio para su transporte entre dispositivos:

- **Matroska (MKV):** contenedor flexible y extensible basado en un formato binario abierto. Voctomix lo emplea como protocolo de transporte interno: las fuentes envían señal I420 y audio PCM S16LE a 48 kHz encapsulados en MKV sobre conexiones

TCP, lo que garantiza la sincronía audio-vídeo y permite multiplexar múltiples pistas en un único flujo sin overhead significativo.

- **MPEG-TS (*Transport Stream*)**: estándar de broadcast diseñado para transmisión con posibilidad de pérdidas. Su robustez frente a errores lo hace idóneo para contribuciones sobre redes no fiables y para la integración con sistemas de distribución broadcast.
- **RTP (*Real-time Transport Protocol*)**: protocolo de transporte en tiempo real utilizado junto con RTSP para la distribución de flujos multimedia. Habitual en cámaras IP profesionales y sistemas SMPTE 2110.

2.3. Distribución Audiovisual

Una vez generada la señal de programa por el mezclador, esta debe distribuirse hasta la audiencia. Esta etapa difiere fundamentalmente de la contribución: mientras que en la contribución se prioriza la calidad y la latencia mínima para preservar la señal sin degradación, en la distribución el objetivo es llegar al máximo número de espectadores con los recursos de red disponibles, lo que obliga a comprimir la señal de forma agresiva y a equilibrar el compromiso entre latencia, calidad y escalabilidad.

2.3.1. Interfaces y protocolos de distribución

RTMP (*Real-Time Messaging Protocol*)

Desarrollado por Macromedia (posteriormente Adobe) sobre TCP, RTMP fue durante más de una década el estándar *de facto* para la ingesta de vídeo en plataformas de streaming como YouTube Live o Twitch. Su latencia típica oscila entre 2 y 5 segundos. Aunque técnicamente obsoleto en el lado del reproductor desde la desaparición de Adobe Flash, sigue siendo el protocolo de ingesta más soportado por encoders y mezcladores de software, incluido Voctomix mediante FFmpeg, y mantiene una tasa de adopción profesional del 58 % en 2025 [10].

SRT (*Secure Reliable Transport*)

SRT es un protocolo de código abierto desarrollado por Haivision que proporciona transmisión de baja latencia (inferior a 1 s) sobre redes no fiables. Incorpora corrección de errores ARQ (*Automatic Repeat Request*) adaptativa y cifrado AES-128/256, lo que lo hace especialmente adecuado para contribuciones y distribuciones sobre Internet pública donde las pérdidas de paquetes y el *jitter* son habituales [11]. Según el *Haivision Broadcast Transformation Report 2025*, SRT es actualmente el protocolo de transporte de vídeo en directo más utilizado entre profesionales, con una tasa de adopción del **77 %** en 2025, frente al 68 % del año anterior, superando ya a RTMP [10].

HLS (*HTTP Live Streaming*)

Desarrollado por Apple, HLS divide el flujo de vídeo en segmentos de fichero de duración fija que se sirven mediante HTTP estándar [12]. Su latencia habitual es de 10–30 segundos en modo estándar, reducible a 2–5 segundos con la extensión *Low-Latency HLS* (LL-HLS). Es el protocolo de reproducción más compatible en 2025, con soporte nativo en prácticamente todos los dispositivos y redes de distribución de contenidos (CDN).

WebRTC (*Web Real-Time Communication*)

Estándar del W3C para comunicación multimedia en tiempo real directamente desde el navegador. Proporciona latencias de 100–500 ms, lo que lo convierte en la única opción viable para aplicaciones verdaderamente interactivas. Su escalabilidad para audiencias masivas es limitada, pero resulta ideal para monitorización remota del sistema de producción, retornos de comunicación entre el equipo técnico y aplicaciones de videoconferencia integradas en el flujo de trabajo.

Tabla 2.3: Comparativa de protocolos de distribución audiovisual en directo.

Protocolo	Latencia	Transporte	Cifrado	Uso principal
RTMP	2–5 s	TCP	No (RTMPS sí)	Ingesta a plataformas
SRT	<1 s	UDP	AES-128/256	Contribución/distribución IP
HLS	2–30 s	HTTP	HTTPS	Reproducción masiva (CDN)
WebRTC	100–500 ms	UDP	DTLS/SRTP	Monitorización, retornos

2.3.2. Códecs de Vídeo

En la etapa de distribución se emplean códecs de alta eficiencia que comprimen la señal para reducir el ancho de banda necesario hasta el espectador final, asumiendo cierta pérdida de calidad a cambio de viabilizar el transporte:

- **H.264 / AVC** (*Advanced Video Coding*, ITU-T H.264): el códec más extendido en la actualidad y la línea de base de la industria. Ofrece buena calidad a tasas de bits moderadas y es soportado por la práctica totalidad de dispositivos, plataformas y hardware de codificación desde 2003. Voctomix lo emplea como códec de salida por defecto mediante FFmpeg [13].
- **H.265 / HEVC** (*High Efficiency Video Coding*): sucesor de H.264 que aproximadamente **duplica la eficiencia de compresión**, permitiendo la misma calidad perceptual a la mitad del bitrate [14]. Su codificación es 5–8 veces más lenta que H.264, lo que lo hace adecuado para streaming en tiempo real cuando se dispone de aceleración hardware (GPU o ASIC). Los dispositivos fabricados tras 2015 disponen en general de soporte de decodificación por hardware.
- **AV1**: códec abierto desarrollado por la Alliance for Open Media que mejora la eficiencia de H.265 en aproximadamente un **50 %** adicional [15]. Su limitación en producción en directo es la velocidad de codificación: AV1 tarda entre 5 y 10 veces más que H.265 en codificar el mismo contenido, lo que restringe su uso en tiempo real a plataformas con aceleración hardware dedicada. YouTube, Netflix y Facebook ya codifican gran parte de su contenido en AV1 para streaming bajo demanda.
- **AAC / Opus**: códecs de audio que acompañan habitualmente a los anteriores en la distribución. AAC (*Advanced Audio Coding*) es el estándar en distribución broadcast y plataformas de streaming; Opus es el códec preferido en aplicaciones WebRTC por su eficiencia a bajas tasas de bits.

Tabla 2.4: Comparativa de códecs de vídeo para distribución en directo.

Códec	Eficiencia vs H.264	Vel. encoding	Compatibilidad	Streaming RT
H.264/AVC	Base	Muy rápido	Universal	Sí
H.265/HEVC	$\times 2$	5–8x más lento	Alta (post 2015)	Sí (con HW)
AV1	$\times 3$	5–10x más lento	Media (creciendo)	Solo con HW

2.3.3. Formatos

El formato contenedor de distribución define cómo se encapsulan los flujos de vídeo y audio codificados para su transporte hasta el reproductor final:

- **MPEG-TS (*Transport Stream*, ISO 13818-1)**: estándar diseñado específicamente para transmisión broadcast con tolerancia a errores. Es el contenedor asociado a HLS, la emisión por satélite y la TDT.
- **MP4 / ISOBMFF (*ISO Base Media File Format*)**: contenedor de referencia para distribución por HTTP progresivo y *streaming* adaptativo (MPEG-DASH, HLS fragmentado). Es el formato de almacenamiento estándar para grabaciones y distribución bajo demanda.
- **FLV (*Flash Video*)**: contenedor legado asociado a RTMP. Aunque técnicamente obsoleto como formato de reproducción desde la retirada de Adobe Flash en 2020, sigue siendo el contenedor utilizado internamente en el protocolo RTMP para la ingesta hacia plataformas de streaming en directo.

3. Diseño e implementación del sistema de producción remota de vídeo

Este capítulo describe el diseño e implementación de Voctomix 2.0: la arquitectura del sistema, sus componentes principales y las decisiones técnicas adoptadas durante el desarrollo. Para cada módulo se exponen los retos encontrados y los mecanismos implementados para resolverlos.

El trabajo comenzó con una fase de exploración sobre Voctomix 1.0 para familiarizarse con la herramienta antes de abordar la versión más reciente. Durante esas primeras semanas se estudió en detalle el fichero de configuración `default.ini`, se comprendió el funcionamiento del protocolo de puertos TCP de entrada de las cámaras (10000, 10001, 10002), se analizaron los flujos de previsualización JPEG que alimentan la interfaz gráfica y se exploró la lógica de los composites disponibles. Esta inmersión en la versión anterior resultó clave para entender los fundamentos del pipeline GStreamer y la comunicación cliente-servidor sobre la que descansa toda la arquitectura, y sirvió de base sólida para dar el salto a Voctomix 2.0 con criterio.

La figura 3.1 ofrece una visión global del sistema completo: las fuentes de señal, el núcleo de procesamiento, las salidas, la interfaz de control y la capa de telemetría. Este diagrama sirve de hilo conductor para los apartados que siguen, cada uno de los cuales profundiza en uno de sus bloques.

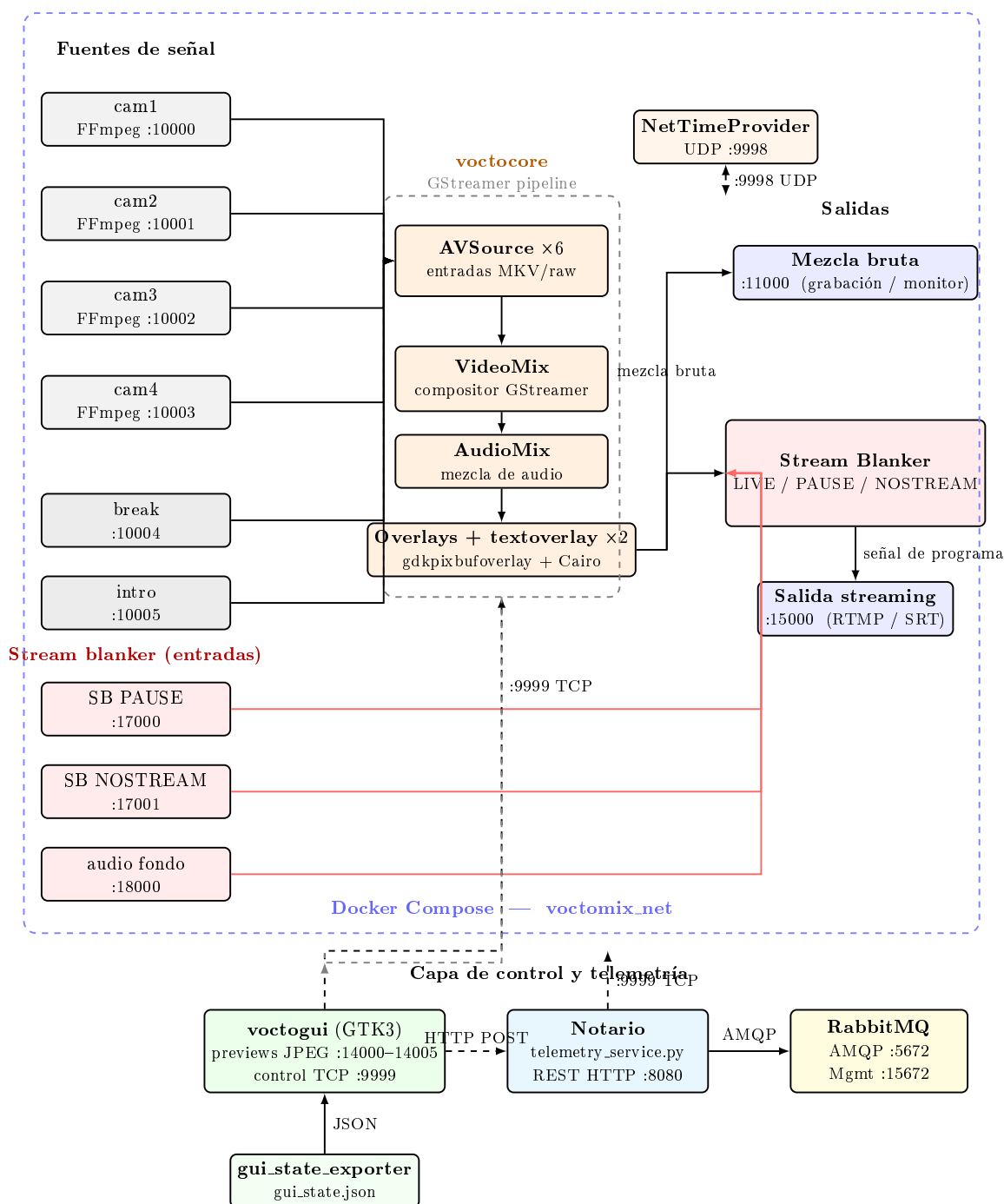


Figura 3.1: Arquitectura del sistema Voctomix 2.0: fuentes de señal (cámaras y auxiliares FFmpeg, entradas del stream blanker), núcleo GStreamer (AVSource, VideoMix, AudioMix, overlays), salidas de mezcla bruta (:11000) y streaming (:15000), capa de control y telemetría (voctogui, servicio de telemetría, RabbitMQ), todo ello orquestado mediante Docker Compose.

La tabla 3.1 recoge los puertos TCP/UDP del sistema y su función.

Tabla 3.1: Puertos del sistema Voctomix 2.0.

Puerto	Protocolo	Función
9998	UDP	Sincronización de reloj de red GStreamer
9999	TCP	Protocolo de control (GUI, scripts, telemetría)
10000	TCP	Entrada cámara 1 (MKV/raw)
10001	TCP	Entrada cámara 2 (MKV/raw)
10002	TCP	Entrada cámara 3 (MKV/raw)
10003	TCP	Entrada cámara 4 (MKV/raw)
10004	TCP	Entrada fuente break (vídeo de pausa)
10005	TCP	Entrada fuente intro (vídeo introductorio)
11000	TCP	Salida mezcla principal en bruto
12000	TCP	Salida previsualización de mezcla (GUI)
14000–14005	TCP	Salidas JPEG comprimidas por fuente (GUI)
15000	TCP	Salida de streaming (tras stream blanker)
17000	TCP	Entrada vídeo stream blanker PAUSE
17001	TCP	Entrada vídeo stream blanker NOSTREAM
18000	TCP	Entrada audio (música de fondo)
8080	HTTP	API REST del servicio de telemetría

3.1. Arquitectura del sistema

3.1.1. voctocore: núcleo multimedia

voctocore es el servidor central del sistema: gestiona todo el procesamiento multimedia, acepta las fuentes de vídeo entrantes, realiza la composición y expone los resultados en distintos puertos TCP. Se inicializa mediante `voctocore/voctocore.py`, que instancia en orden el **Pipeline** central (grafo GStreamer), el **ControlServer** TCP y el proveedor de tiempo de red GStreamer (**NetTimeProvider**, puerto UDP 9998). El **Pipeline** construye dinámicamente la cadena GStreamer a partir de la configuración INI: crea una instancia de **AVSource** por cada fuente declarada, instancia el **VideoMix** y el **AudioMix**, conecta el **StreamBlanker** y abre los puertos de salida.

El **ControlServer** expone el **protocolo de control** en el puerto TCP 9999: un protocolo de texto plano en el que cada comando tiene la forma `<verbo>_<objeto> [parámetros]` (p.ej. `set_video_a cam2` o `set_composite_mode pip`). El servidor transmite por *broadcast* cada cambio de estado a todos los clientes conectados simultáneamente, de forma que la interfaz gráfica, los scripts de automatización y el servicio de telemetría mantienen siempre una vista coherente del sistema sin necesidad de consultar el núcleo activamente.

3.1.2. voctogui: interfaz gráfica de control

voctogui es el cliente de control: implementado en PyGTK3, proporciona al realizador la interfaz visual para operar el sistema en tiempo real. Se conecta al núcleo mediante

un socket TCP no bloqueante gestionado con `GObject.io_add_watch`, de forma que los eventos de red no bloquean el hilo principal de GTK. La GUI recibe los flujos de previsualización de cada fuente en formato JPEG comprimido (puertos 14000–14005) para minimizar el consumo de ancho de banda, mientras que el monitor principal recibe la mezcla final en bruto (puerto 11000).

3.2. Fuentes de señal de cámaras

El sistema acepta hasta cuatro fuentes de cámara simultáneas (cam1–cam4), que constituyen las entradas principales del mezclador. Cada cámara envía su señal al núcleo mediante una conexión TCP en formato Matroska con vídeo en bruto (I420) y audio PCM S16LE a 48 kHz estéreo.

3.2.1. Conexión de las fuentes

Cada fuente de cámara se conecta al puerto TCP correspondiente de voctocore:

- **cam1** → puerto 10000
- **cam2** → puerto 10001
- **cam3** → puerto 10002
- **cam4** → puerto 10003

El envío de la señal puede realizarse con cualquier herramienta capaz de producir un flujo Matroska sobre TCP. En el entorno de desarrollo y pruebas, las fuentes de cámara se simulan mediante scripts FFmpeg que generan vídeo de prueba (barras de color o *test card* con reloj en tiempo real). En un despliegue real con cámaras físicas, la señal se captura habitualmente mediante tarjetas de captura y se reencapsula en Matroska/TCP con FFmpeg.

Los scripts de prueba se encuentran en el directorio `example-scripts/ffmpeg/source-testvideo-as-c`. En el escenario Docker, cada cámara es un contenedor independiente que comparte el *network namespace* de voctocore (directiva `network_mode: service:voctocore`), lo que les permite conectarse directamente por `localhost` sin overhead de red virtual.

3.2.2. Previsualización en la interfaz gráfica

Una vez conectadas, las fuentes de cámara aparecen automáticamente en el panel de previsualizaciones de voctogui. La interfaz muestra cuatro miniaturas JPEG de las fuentes activas, recibidas desde los puertos 14000–14003 a una resolución reducida (1024×576) para minimizar el consumo de ancho de banda entre el servidor y el cliente de control. Cada miniatura incluye:

- Un indicador visual del nombre de la fuente (cam1, cam2, cam3, cam4).
- Botones de selección para asignar la fuente al bus A o al bus B del mezclador.
- Un indicador del nivel de audio asociado a esa fuente.

El realizador selecciona la fuente activa haciendo clic sobre la miniatura correspondiente, lo que envía el comando `set_video_a <fuente>` o `set_video_b <fuente>` al núcleo

mediante el protocolo de control TCP.

3.2.3. Requisitos de formato de la señal de entrada

Durante el desarrollo se comprobó que voctocore es estricto con los metadatos del flujo de vídeo entrante. Al intentar inyectar un archivo `.mp4` convencional como fuente de prueba, el núcleo rechazaba la conexión sin emitir un mensaje de error claro. Tras el diagnóstico se identificaron dos requisitos obligatorios que toda fuente debe cumplir:

1. **Pista de audio obligatoria:** voctocore exige que cada flujo Matroska contenga tanto vídeo como audio. Los vídeos sin audio deben complementarse con una pista de silencio generada en tiempo real por FFmpeg mediante el filtro `-f lavfi -i aevalsrc=0`. Sin este ajuste, el motor GStreamer cierra la conexión en el *handshake* inicial.
2. **Colorimetría televisiva (BT.709):** el núcleo exige que los metadatos del flujo declaren el espacio de color estándar de alta definición (`colorimetry=bt709`) y el rango de luz televisivo (`tv range`). Los archivos grabados con cámaras convencionales o editados con software de consumo suelen emplear el rango `pc` (full range). Para traducirlos, se aplican los filtros FFmpeg `scale=out_range=tv` y `colorspace=bt709`, que recalculan los valores de píxel al vuelo sin degradación visual perceptible.

Estos requisitos quedan encapsulados en los scripts de prueba `example-scripts/ffmpeg/source-testv` que sirven como referencia para la integración de cámaras físicas reales mediante tarjetas de captura.

3.3. Fuentes de señal auxiliares

Además de las cuatro fuentes de cámara, el sistema incorpora varias fuentes de señal auxiliares que permiten gestionar el estado de la emisión en momentos en los que la mezcla principal no debe ser visible para el espectador, así como una entrada de audio independiente para música de fondo.

3.3.1. Fuente break e intro

`break` e `intro` son fuentes de vídeo pregrabadas que el núcleo trata exactamente igual que una cámara adicional. Se conectan a los puertos 10004 (`break`) y 10005 (`intro`) mediante procesos FFmpeg gestionados desde los scripts de arranque:

- **break:** vídeo en bucle empleado como pantalla de pausa animada durante los descansos del evento. FFmpeg lo envía con la opción `-stream_loop -1` para que la fuente nunca quede en negro, independientemente de la duración del archivo. El realizador puede seleccionarlo como fuente activa del mezclador o bien activarlo automáticamente a través del mecanismo de stream blanker.
- **intro:** vídeo introductorio que se reproduce al inicio del evento o entre sesiones. A diferencia del `break`, la `intro` *no* se reproduce en bucle continuo, sino que se dispara una sola vez cuando el realizador selecciona esa fuente en la interfaz gráfica.

Mecanismo de disparo de la intro (*vigilante*)

El comportamiento de disparo único de la intro plantea un reto arquitectónico: la GUI de voctogui no puede ejecutar comandos externos sin bloquear el hilo principal de GTK. Para resolverlo sin modificar el núcleo del sistema, se implementó un proceso independiente denominado `vigilante_intro.py`.

Este script actúa como un cliente silencioso del puerto de control TCP (9999): escucha el flujo de notificaciones de estado que voctocore difunde por *broadcast* y, cuando detecta el mensaje `video_status intro` (el realizador ha seleccionado la fuente intro), ejecuta inmediatamente el script `lanzar_intro.sh` mediante `subprocess.Popen` sin buffering para garantizar una reacción instantánea.

Esta arquitectura sigue la filosofía de separación de responsabilidades de Voctomix: el núcleo solo sabe mezclar señales; los automatismos de producción se implementan como clientes externos que reaccionan a su protocolo de estado. Si el vigilante fallara, el sistema de mezcla continuaría operando con plena normalidad.

3.3.2. Entrada de audio

El sistema incorpora una entrada de audio independiente en el puerto 18000, pensada para inyectar música de fondo durante los estados de pausa o fuera de emisión. Esta entrada acepta un flujo Matroska de sólo audio (PCM S16LE, 48 kHz, estéreo) y su nivel de salida puede controlarse desde el protocolo de control.

3.3.3. Stream Blanker: estados de emisión

El stream blanker es el mecanismo que permite interrumpir la señal de programa hacia el exterior sin interrumpir el flujo de trabajo interno del realizador. Se implementa en `voctocore/lib/streamblanker.py` como un elemento GStreamer adicional interpuesto entre la mezcla y el puerto de salida de streaming (puerto 15000).

El sistema define tres estados de emisión, controlables desde la barra de herramientas de voctogui:

- **LIVE:** se emite la mezcla activa producida por voctocore. El realizador controla en todo momento lo que ve el espectador.
- **PAUSE:** se emite el vídeo de pausa procedente del puerto 17000 (fuente `stream-blanker-pause`), acompañado de la música de fondo del puerto 18000. Se usa durante los descansos del evento para que los espectadores vean una pantalla animada mientras el equipo de producción prepara la siguiente sesión.
- **NOSTREAM:** se emite un vídeo de “fuera de emisión” procedente del puerto 17001 (fuente `stream-blanker-nostream`), indicando al espectador que la transmisión ha finalizado o aún no ha comenzado.

La transición entre estados se realiza modificando los pesos del compositor de GStreamer en tiempo real, evitando cortes bruscos en el stream de salida hacia el encoder. Desde la interfaz gráfica, el realizador dispone de tres botones (LIVE, PAUSE, NOSTREAM) en la barra de herramientas (`voctogui/lib/toolbar/streamblank.py`).

3.4. Composites

Los composites definen cómo se combinan visualmente las dos fuentes de vídeo activas (bus A y bus B) en el fotograma de salida. Voctomix implementa los composites mediante el elemento GStreamer `compositor`, que sitúa y escala cada fuente de acuerdo con los parámetros geométricos definidos en la sección `[composites]` del fichero de configuración INI. La modificación de estos parámetros en tiempo real permite transiciones suaves entre composites sin generar fotogramas corruptos.

Los composites disponibles en Voctomix 2.0 son los siguientes:

La figura 3.2 ilustra la disposición visual de las dos fuentes (A en naranja, B en azul) en cada uno de los composites principales.

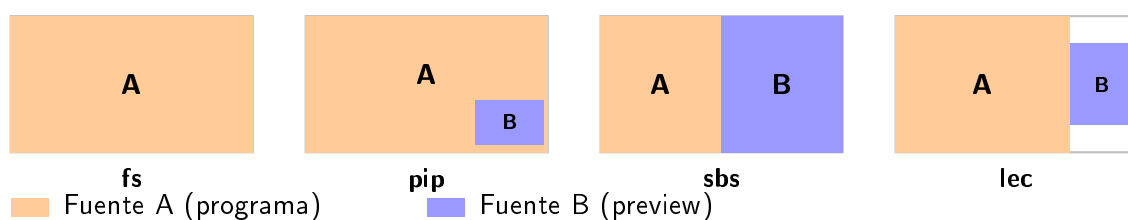


Figura 3.2: Disposición visual de las fuentes A y B en los cuatro composites principales de Voctomix 2.0: pantalla completa (**fs**), imagen en imagen (**pip**), lado a lado (**sbs**) y conferencia (**lec**).

3.4.1. Full Screen (**fs**)

La fuente A ocupa toda la pantalla y la fuente B queda oculta ($\alpha = 0$). Es el modo por defecto de una realización convencional: el realizador selecciona una fuente y la emite a pantalla completa. Resulta adecuado para la emisión de una ponencia individual, una demostración en pantalla o cualquier situación en la que una única fuente deba ocupar todo el encuadre.

3.4.2. Picture in Picture (**pip**)

La fuente A se muestra a pantalla completa mientras la fuente B aparece reducida en una esquina del encuadre (típicamente la esquina inferior derecha), a una escala aproximada del 27 % del ancho del fotograma. Es el composite clásico de “ponente con presentación”: la cámara del ponente ocupa la pantalla completa y las diapositivas aparecen en la esquina, o viceversa. Puede configurarse con `mirror=true` para que la fuente A se invierta horizontalmente, útil cuando la cámara tiene la imagen especular por la disposición del equipo.

3.4.3. Side by Side (**sbs**)

Ambas fuentes se muestran a igual tamaño, situadas una junto a la otra y ocupando cada una aproximadamente la mitad del fotograma. Es útil para comparaciones visuales, debates con dos participantes o para mostrar simultáneamente la cámara del ponente y sus diapositivas con el mismo peso visual.

3.4.4. Lecture (lec)

La fuente A ocupa la mayor parte del encuadre (aproximadamente el 75 %) mientras que la fuente B aparece reducida en la parte derecha. Este composite está específicamente diseñado para retransmisiones de ponencias o clases: las diapositivas (fuente A) dominan el encuadre y la cámara del ponente (fuente B) aparece en un recuadro lateral de menor tamaño, manteniendo siempre la legibilidad del contenido proyectado.

La variante `lec_43` está concebida para acomodar contenido con relación de aspecto 4:3 (como proyectores antiguos o capturas de escritorio con resolución 1024×768) en un encuadre 16:9. En la versión original del repositorio, esta variante aplicaba un recorte del 31 % mediante el parámetro `crop-a=0.125/0` que destruía el encuadre completo cuando la fuente era nativa 16:9. Se corrigió redefiniendo los parámetros de posición y escala en la sección `[composites]` del fichero INI para eliminar el recorte y ajustar la ventana secundaria de forma que encaje perfectamente a la derecha sin desbordarse del fotograma.

3.4.5. Mirror

Aunque no es un composite independiente sino una propiedad disponible en varios de los anteriores (`mirror=true` en `pip`, `lec` y `fs-pip`), el efecto espejo permite invertir horizontalmente la fuente A o B en el composite. Esto es necesario cuando la señal de cámara llega especularmente invertida, como puede ocurrir con ciertos modelos de cámaras web o cuando la cámara está orientada de forma atípica. Al activar el mirror desde el fichero de configuración, el núcleo aplica la inversión directamente en el compositor de GStreamer sin coste adicional.

3.5. Transiciones

Las transiciones controlan cómo se realiza el cambio de una fuente o composite a otro en el flujo de programa. Voctomix 2.0 ofrece tres modos de transición que el realizador activa desde la barra de herramientas de la interfaz gráfica.

3.5.1. Cut (corte instantáneo)

El botón **Cut** realiza una transición instantánea: en el siguiente fotograma tras la pulsación, la fuente de programa pasa a ser la fuente de preview. No existe ningún efecto intermedio. Es la transición estándar en producción broadcast: el corte imita el montaje cinematográfico y resulta imperceptible cuando el *timing* es correcto.

3.5.2. Trans (transición suave)

El botón **Trans** realiza una transición animada entre el estado actual y el nuevo estado de composite o fuente, interpolando los parámetros geométricos y de transparencia durante un tiempo configurable (por defecto 750 ms, definido en la sección `[transitions]` del fichero INI). La transición se implementa modificando progresivamente los valores de los pads del compositor de GStreamer.

Voctomix 2.0 define múltiples trayectorias de transición según el estado de origen y destino (por ejemplo, de `fs` a `pip` la transición pasa por el composite intermedio `fs-pip` antes

de llegar a `pip`), garantizando que la animación sea visualmente coherente independientemente del cambio solicitado.

3.5.3. Retake

El botón **Retake** permite al realizador deshacer el último cambio de fuente o composite y volver al estado inmediatamente anterior sin interrumpir la señal de programa. Su funcionamiento es equivalente a un Cut inverso: en el siguiente fotograma, el sistema intercambia programa y preview, restaurando la fuente que estaba en antena antes del cambio. Esta función resulta especialmente útil cuando el realizador realiza un corte erróneo y necesita corregirlo de inmediato sin generar un segundo corte visible en la emisión.

3.6. Overlays, logos y texto dinámico

Voctomix 2.0 incorpora un sistema completo de superposición de elementos gráficos sobre la señal de vídeo: logos corporativos, overlays PNG con canal alfa y texto dinámico renderizado en tiempo real. Todos estos elementos se gestionan desde la interfaz gráfica sin interrumpir el pipeline de procesamiento.

3.6.1. Logos

El sistema soporta hasta dos logos simultáneos, superpuestos en posiciones fijas del encuadre definidas mediante la configuración INI:

- **logo1**: posicionado en la esquina inferior derecha del fotograma.
- **logo2**: posicionado en la esquina superior izquierda del fotograma.

Los ficheros PNG de los logos se definen en las secciones `[logo1]` y `[logo2]` del fichero de configuración, permitiendo adaptar la identidad visual del evento sin modificar el código. La visibilidad de cada logo se puede activar o desactivar individualmente desde la interfaz gráfica.

3.6.2. Overlays PNG y texto dinámico

Los overlays son imágenes PNG con canal alfa que se superponen sobre la señal de vídeo mediante el elemento GStreamer `gdkpixbufoverlay`. El módulo `voccore/lib/overlay.py` gestiona el ciclo de vida completo de un overlay: carga del fichero PNG, fundido de entrada (*blend-in*), mantenimiento en pantalla y fundido de salida (*blend-out*). El tiempo de fundido se controla mediante el parámetro `blend-time` en la sección `[overlay]` del fichero INI (en milisegundos).

Durante el desarrollo se identificó una limitación del elemento `gdkpixbufoverlay` de GStreamer: está diseñado para leer archivos de imagen estáticos desde una ruta del sistema de ficheros, sin soporte nativo para actualizaciones en tiempo real. La solución adoptada consiste en regenerar el fichero PNG en disco con el nuevo texto renderizado mediante Cairo y forzar la recarga del elemento GStreamer mediante el comando `set_overlay` del protocolo de control. Este mecanismo permite actualizar los rótulos en caliente durante la emisión con una latencia inferior a un fotograma.

3.6.3. Rotulación dinámica de ponente

El sistema de rotulación permite al operador escribir el nombre del ponente o el título de la sesión en un campo de texto de la interfaz gráfica y enviarlo al núcleo para su superposición inmediata sobre el vídeo. La cadena de procesamiento completa es la siguiente:

1. El operador escribe el texto en el campo `GtkEntry` del panel de inserciones de `voctogui`.
2. Al pulsar *Enter* o el botón **INSERT**, el módulo `overlay.py` de la GUI solicita a Cairo la renderización del texto sobre la plantilla PNG activa.
3. La imagen resultante se escribe en disco y se envía al núcleo mediante el comando `TCP set_overlay <ruta_fichero>`.
4. `votocore` recarga `gdkpixbufoverlay` con la nueva imagen, que aparece en antena con el efecto de fundido configurado.

3.6.4. Sistema de doble rotulación

Para aumentar la versatilidad del sistema de grafismo, se implementó un segundo motor de rotulación independiente que permite mostrar simultáneamente dos textos en posiciones distintas del encuadre. Los cambios realizados abarcan las cuatro capas del sistema:

- **Motor de vídeo (`videomix.py`):** se inyectó un segundo nodo `textoverlay` en el pipeline `GStreamer` (`rotulo_dinamico_2`), configurado con alineación horizontal opuesta al primero para evitar solapamientos. Se implementó el método `set_dynamic_text_2` para actualizar sus propiedades en caliente.
- **Protocolo de control (`commands.py`):** se amplió el servidor TCP con el comando `set_text2` y se configuró la notificación `text2_updated` para que cualquier cliente conectado sea informado del cambio.
- **Interfaz gráfica (`voctogui.ui` y `voctogui.css`):** se creó un segundo contenedor `GtkBox` en el panel de inserciones, situado junto al primero. Los nuevos selectores CSS (`#tfg-entry-2`, `#tfg-btn-2`) heredan la estética corporativa oscura y el efecto de iluminación amarilla al activarse.
- **Lógica de foco y teclado (`overlay.py`):** se perfeccionó el algoritmo de intercepción de atajos de teclado a nivel de ventana. El sistema evalúa dinámicamente qué campo de texto tiene el foco (`is_focus()`) para procesar correctamente las teclas críticas (Espacio, Retroceso, *Enter*), evitando colisiones con los atajos globales del mezclador.

3.6.5. Botón UPDATE: recarga dinámica de recursos

El botón **UPDATE** del panel de overlays permite recargar el directorio de recursos gráficos durante la emisión. Esta funcionalidad es crítica en entornos de directo: si el equipo de grafismo corrige un error ortográfico en una plantilla PNG o añade un nuevo rótulo de último minuto, el operador puede incorporarlo al sistema sin interrumpir la señal de vídeo ni reiniciar ningún proceso.

3.6.6. Auto-off de overlays

La funcionalidad *auto-off* resuelve un problema frecuente en producción en directo: el operador realiza un corte de cámara olvidando retirar el overlay activo, que queda superpuesto sobre la nueva fuente de forma inapropiada (por ejemplo, el nombre de un ponente apareciendo sobre la imagen de otro).

Cuando la opción `auto-off = true` está activa en la configuración, el módulo `overlay.py` registra un *listener* sobre el evento `video_status`. Cada vez que la fuente de vídeo activa cambia, el listener inicia automáticamente la secuencia de fundido de salida del overlay activo. Si hay texto dinámico superpuesto, éste se elimina junto al overlay gráfico, garantizando que ningún elemento de rotulación quede huérfano en pantalla.

El comportamiento puede configurarse mediante:

- `auto-off = true/false`: activa o desactiva la funcionalidad.
- `user-auto-off = true/false`: expone un botón de alternancia en la GUI para que el operador pueda activar/desactivar el auto-off en tiempo real.
- `blend-time = <ms>`: tiempo de fundido de salida en milisegundos.

3.7. Señal de programa final

Una vez procesada la mezcla — con el composite activo, las transiciones aplicadas y los overlays superpuestos — voctocore emite la señal de programa final de forma simultánea por dos puertos:

- **Puerto 11000**: salida de la mezcla en bruto (vídeo sin comprimir en formato I420), destinada a herramientas de grabación o encoders externos como FFmpeg. Esta salida refleja siempre el estado real de la mezcla, independientemente del estado del stream blanker.
- **Puerto 15000**: salida de streaming, que pasa previamente por el stream blanker. Cuando el estado es LIVE, esta salida es idéntica al puerto 11000; cuando el estado es PAUSE o NOSTREAM, emite el vídeo alternativo correspondiente en lugar de la mezcla real.

El encoder de salida (típicamente FFmpeg) se conecta al puerto 15000 y produce el flujo H.264/AAC que se ingesta en la plataforma de distribución (RTMP/SRT hacia YouTube Live, Twitch u otras plataformas). La separación entre los puertos 11000 y 15000 garantiza que la grabación local refleje siempre la mezcla real del realizador, mientras que la audiencia remota solo recibe la señal que el operador autoriza explícitamente a través del stream blanker.

3.8. Personalización de la interfaz gráfica

Una vez que el pipeline de procesamiento multimedia estaba operativo y las funciones principales del sistema eran estables, se abordó la transformación visual y operativa de voctogui. La interfaz original presentaba un aspecto genérico, con fondo blanco heredado del tema del sistema operativo y sin iconografía coherente. Esta personalización se realizó

de forma progresiva a lo largo del desarrollo, en paralelo a la incorporación de nuevas funcionalidades, con el objetivo de convertirla en una herramienta de producción profesional lista para su uso en eventos reales.

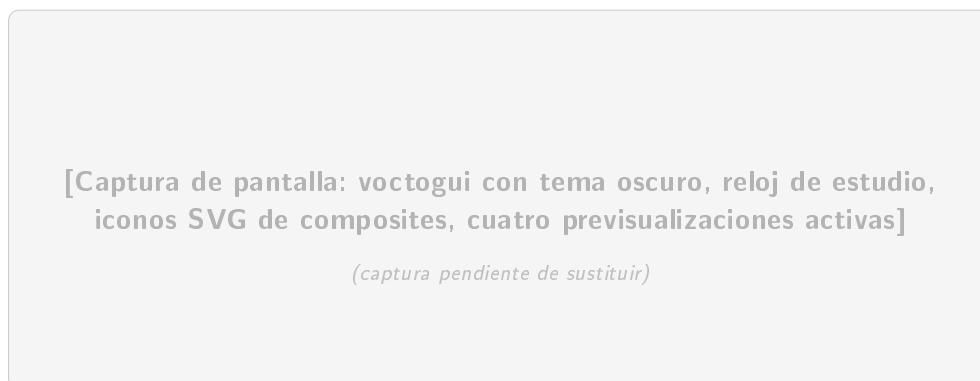


Figura 3.3: Interfaz gráfica voctogui tras la personalización: tema oscuro CSS (#252a2c), iconos SVG de composites, reloj de estudio y cuatro paneles de previsualización activos.

3.8.1. Tema oscuro y hoja de estilos

Se inyectó una hoja de estilos CSS (`voctogui.css`) que fuerza un fondo #252a2c y texto claro en todos los paneles. Los botones activos se iluminan en amarillo (#ddbb00), unificando el lenguaje visual de toda la interfaz y proporcionando al realizador una retroalimentación visual inmediata sobre el estado de cada control.

3.8.2. Iconografía SVG

Los botones de composite y de control de emisión se vincularon a iconos vectoriales .svg, declarados en las secciones `[toolbar.composites]` y `[toolbar.mix]` del fichero de configuración `default-config.ini`. El uso de SVG garantiza que los iconos se escalan sin pérdida de calidad a cualquier resolución de pantalla.

3.8.3. Reloj de estudio

Se integró el widget `StudioClock` en la parte inferior del panel de cámaras mediante el diseño XML de la interfaz (`voctogui.ui`), envuelto en un `GtkAspectFrame` para preservar las proporciones del panel de vídeo. El reloj muestra la hora del sistema en tiempo real, indispensable para la coordinación de escaleta en producciones en directo: el realizador puede sincronizar los cortes de cámara con los tiempos marcados en el guión sin apartar la vista del monitor de mezcla.

3.9. Sistema de telemetría

El sistema de telemetría de Voctomix 2.0 permite registrar en tiempo real todos los eventos que ocurren durante una sesión de producción: cambios de fuente activa, cambios de composite, activaciones del stream blanker, carga y descarga de overlays, niveles de audio, y métricas de rendimiento del servidor. Esta información resulta valiosa tanto para la monitorización durante el directo como para la auditoría posterior de la sesión. El módulo de todo lo que ocurre en la mesa de mezclas.

3.9.1. Arquitectura del servicio de telemetría

El servicio de telemetría (`example-scripts/ffmpeg/telemetry_service.py`) es un proceso Python independiente que no requiere privilegios especiales sobre el núcleo. Su arquitectura se articula en cinco capas funcionales:

Intercepción del núcleo por TCP

El servicio abre una conexión de socket TCP al puerto 9999 de voctocore, comportándose como un cliente de control más. Escucha en un hilo secundario (`threading`) los mensajes del protocolo interno, filtrando los eventos de estado relevantes: `video_status` (fuente en previo y en programa), `composite_mode` (distribución de pantalla activa), y `stream_status` (LIVE, PAUSE, NOSTREAM).

El handshake inicial incluye el comando `get_config`, que devuelve en un bloque JSON los parámetros del pipeline: resolución, framerate, formato de píxel (`yuv420p`), frecuencia de muestreo de audio y número de canales. Estos datos se inyectan en el campo `stream_info` de cada entrada del log, dotando al registro de validez técnica para análisis de rendimiento.

Extracción del estado de la interfaz gráfica

Se identificó un reto arquitectónico crítico: voctocore desconoce el estado de los elementos personalizados de la GUI (rótulos dinámicos, niveles de los *faders* de audio) porque estos se gestionan puramente en la capa gráfica. Para capturarlos sin modificar el núcleo, se desarrolló el módulo `gui_state_exporter.py`, integrado en voctogui.

Este módulo lee constantemente el estado de los widgets GTK mediante `GLib.idle_add` (para garantizar la seguridad de hilos) y exporta un fichero local `gui_state.json` con los niveles de audio por cámara (`cam1_db`, etc.) y el estado de la superposición de textos y logos. El servicio de telemetría consume este fichero JSON para completar la vista unificada del sistema.

Fusión de estados y doble lógica de registro

El servicio unifica los datos del núcleo (TCP) y de la interfaz (JSON local) mediante un sistema de fusión protegido por cerrojos de hilo (`threading.Lock()`) para evitar condiciones de carrera. El registro sigue una doble vía temporal:

- **Por evento (CHANGE):** en cuanto se detecta cualquier variación de estado, la entrada se escribe inmediatamente en el log con su marca de tiempo. Esta modalidad captura cada corte de cámara o cambio de composite en el instante exacto en que ocurre.
- **Por latido (STATE):** un hilo paralelo escribe el estado completo del sistema cada 5 segundos como medida de redundancia y control de *uptime*, independientemente de que haya habido cambios.

Telemetría técnica: métricas de rendimiento

Para dotar al log de utilidad operativa, el servicio registra en cada entrada un bloque `performance` con las siguientes métricas del servidor:

- **Ancho de banda por cámara (`bandwidth_mbps`):** el vídeo en bruto (Raw YUV420p) no varía con el contenido visual. El valor se calcula matemáticamente como Mbps =

$\frac{\text{Ancho} \times \text{Alto} \times \text{FPS} \times 1,5}{1\,000\,000}$. Para 1080p25 el resultado es aproximadamente 622 Mbps por fuente, lo que evidencia que todo el tráfico de vídeo debe circular en bucle local (localhost o red interna Docker) sin atravesar interfaces físicas convencionales.

- **Uso de CPU** (`cpu_usage_percent`): leído de `/proc/stat`, permite al operador verificar que el servidor trabaja por debajo del umbral de saturación. Si la CPU supera el 80 %, GStreamer comienza a descartar fotogramas, degradando la emisión.
- **Memoria RAM** (`ram_usage_percent`, `ram_available_mb`): leído de `/proc/meminfo`. Un consumo bajo y estable confirma que los pipelines GStreamer procesan y liberan fotogramas en tiempo real sin acumular búferes en memoria.
- **Tiempos de sesión** (`uptime`, `session_duration`): `uptime` indica el tiempo de encendido del servidor; `session_duration` actúa como cronómetro del evento y permite correlacionar los cortes registrados en el JSON con el código de tiempo del archivo de grabación final.

Formato de salida: JSON Lines

Cada entrada del log se escribe como un objeto JSON completo en una línea del fichero `registroN.jsonl` (*JSON Lines*). Este formato permite leer el log de forma incremental con herramientas estándar (jq, Python, scripts de análisis) sin necesidad de cargar el fichero completo en memoria, lo que resulta práctico en sesiones de larga duración.

Un ejemplo de entrada de tipo CHANGE tras un corte de cámara tendría la siguiente estructura:

```
{
  "timestamp": "2025-03-28T20:14:32.105Z",
  "type": "CHANGE",
  "video": {
    "program": "cam2",
    "preview": "cam1"
  },
  "composite": "fs",
  "stream": "live",
  "overlay": {
    "active": false,
    "text": ""
  },
  "audio": {
    "cam1_db": -12.0,
    "cam2_db": 0.0
  },
  "performance": {
    "cpu": 34.2,
    "ram_pct": 8.7,
    "bandwidth_mbps": 622.1
  },
  "stream_info": {
    "width": 1920,
    "height": 1080,
    "fps": 25,
    "pix_fmt": "yuv420p"
  }
}
```

Autodescubrimiento de red

Para que el log identifique el origen de los comandos en escenarios distribuidos (dos PCs, Docker), el servicio implementa una función de autodescubrimiento de red en dos niveles:

1. **Nivel contenedor (prioridad)**: si existe la variable de entorno `MI_IP_REAL` (inyectada por Docker Compose), el servicio la utiliza directamente como IP de la interfaz LAN real del servidor.
2. **Nivel nativo (fallback)**: si la variable no existe, el script activa un socket UDP hacia 8.8.8.8 sin llegar a realizar la conexión efectiva, interrogando a la tabla de enrutamiento del kernel para identificar qué interfaz física tiene salida a Internet y obteniendo así la IP real de la LAN de forma automática.

3.9.2. API REST y mensajería AMQP

El servicio expone los datos de telemetría en dos canales complementarios:

- **API REST HTTP (puerto 8080):** implementada con `http.server.HTTPServer` de la librería estándar de Python, sin dependencias externas. Devuelve el estado actual completo de la sesión en formato JSON. Para prevenir bloqueos de puerto en reinicios rápidos se activa la opción de socket `SO_REUSEADDR`, equivalente al parámetro `allow_reuse_address = True` de Python.
- **Mensajería AMQP (RabbitMQ):** cada cambio de estado se publica como un mensaje JSON en el exchange `vocotmix_events` del broker RabbitMQ, permitiendo que cualquier consumidor externo (sistema de streaming, dashboard web, automatización) se suscriba a los eventos sin acoplarse directamente al el servicio de telemetría ni a `vocotcore`.

3.9.3. RabbitMQ como broker de mensajería

RabbitMQ es el broker de mensajería AMQP que centraliza los eventos de telemetría. En el despliegue Docker se ejecuta como un contenedor independiente accesible en el puerto 5672 (protocolo AMQP) y 15672 (consola web de administración). Las credenciales por defecto son `vocotmix/vocotmix123`.

La integración con RabbitMQ permite:

- Mantener un registro persistente de todos los eventos de la sesión con marca de tiempo, para análisis posterior.
- Integrar Voctomix con sistemas externos de monitorización, *logging* centralizado (ELK Stack, Grafana) o automatización mediante consumidores de la cola.
- Verificar el correcto funcionamiento del sistema durante las pruebas, comprobando que cada acción del realizador genera el evento de telemetría correspondiente.

Un ejemplo de consumidor de la cola RabbitMQ se encuentra en `rabbit_consumer_example.py`, que sirve como punto de partida para integrar Voctomix con sistemas externos.

La arquitectura desacoplada del servicio de telemetría, conectado al núcleo como un cliente TCP ordinario sin privilegios especiales, garantiza que su eventual fallo no afecte al procesamiento multimedia principal: el sistema de producción opera con plenas garantías aunque el servicio de telemetría no esté disponible (*single point of failure* libre).

3.9.4. Evolución del protocolo de telemetría: de UDP a HTTP

En el escenario de dos PCs, el servicio de telemetría necesita recibir los datos de estado de la GUI (niveles de audio, estado de overlays) desde el PC del realizador a través de la red. La implementación inicial utilizó UDP (`socket.SOCK_DGRAM`), que presenta dos fallos críticos en entornos Wi-Fi:

- **Falta de fiabilidad:** UDP es *fire-and-forget*; los paquetes de estado pueden perderse sin que el emisor lo detecte, dejando el servicio y la GUI desincronizados.
- **Bloqueos del kernel (Errno 98):** al reiniciar los scripts rápidamente durante el desarrollo, el kernel mantenía el puerto UDP en estado `TIME_WAIT`, impidiendo que

el servicio volviera a enlazar el puerto hasta que expirase el tiempo de gracia.

La solución fue migrar a un modelo **HTTP/TCP**: el servicio de telemetría actúa como servidor HTTP en el puerto 8080, y el `gui_state_exporter.py` del PC del realizador envía los datos de estado mediante peticiones HTTP POST con cuerpo JSON. TCP garantiza la entrega secuencial de los datos, y HTTP proporciona un estándar robusto para el intercambio de estructuras JSON. Adicionalmente, se estableció un *timeout* de 1 segundo en el cliente (`urlopen(req, timeout=1.0)`) para que la GUI nunca se bloquee si el PC del servidor no está disponible.

3.10. Contenerización Docker

La contenerización de Voctomix 2.0 persigue un doble objetivo: garantizar la reproducibilidad del sistema en cualquier equipo eliminando el problema de dependencias y versiones de sistema operativo, e implantar un mecanismo de aislamiento de procesos que mejore la resiliencia ante fallos en producción. El despliegue se articula en dos ficheros: un **Dockerfile** que define la imagen base del sistema y un `docker-compose.yml` que orquesta el conjunto de servicios.

El código original del repositorio (año 2016) contenía un **Dockerfile** basado en Ubuntu 15.10 (Wily) con dependencias incompatibles con los repositorios actuales y una imagen de base `c3voc/voctomix` que ya no existe en Docker Hub. Se eliminó completamente ese material y se redactó una arquitectura moderna desde cero.

3.10.1. Dockerfile

La imagen se construye sobre `ubuntu:22.04 LTS`, garantizando soporte a largo plazo y parches de seguridad. La instalación de dependencias se realiza en modo desatendido (`DEBIAN_FRONTEND=noninteractive`) en una sola capa para minimizar el tamaño de la imagen:

- GStreamer 1.0 con sus plugins *base*, *good*, *bad* y *ugly*.
- FFmpeg, para la inyección de fuentes de vídeo.
- *Bindings* Python de GObject (`python3-gi`), la biblioteca Cairo para renderizado de overlays, y la librería AMQP (`pika`) para la integración con RabbitMQ.
- Utilidades de red (`netcat`, `iproute2`) necesarias para el *healthcheck* y el autodescubrimiento de IP.
- Librerías matemáticas `scipy` y `numpy`, necesarias para el cálculo de curvas B-Spline en las transiciones animadas.

El directorio de trabajo del contenedor es `/opt/voctomix` y el punto de entrada es `votocore.py`.

Un *healthcheck* basado en `nc -zw1 localhost 9999` permite a Docker Compose determinar cuándo el núcleo está listo para aceptar conexiones antes de lanzar los servicios dependientes, evitando condiciones de carrera durante el arranque.

3.10.2. Docker Compose y patrón *Shared Network Namespace*

La arquitectura de Voctomix exige que las fuentes de cámara (procesos FFmpeg independientes) se conecten al servidor local mediante `localhost`. En Docker, cada contenedor tiene por defecto su propio `localhost` aislado, lo que rompería todas las conexiones TCP de entrada al núcleo.

En lugar de reescribir las direcciones IP en todo el código fuente, se adoptó el patrón *Shared Network Namespace*: los contenedores de las cámaras y el servicio de telemetría declaran `network_mode: "service:votocore"`, lo que fusiona sus interfaces de red virtuales con la del contenedor del núcleo. Desde el punto de vista de Linux, todos estos procesos comparten la misma tarjeta de red, por lo que `localhost` es el mismo en todos ellos. Este patrón es habitual en arquitecturas de *sidecar* y permite mantener el código de los scripts de cámara sin ninguna modificación.

El fichero `docker-compose.yml` define los siguientes servicios organizados en una red *bridge* privada (`votomix_net`):

Tabla 3.2: Servicios definidos en `docker-compose.yml`.

Servicio	Imagen base	Puertos expuestos	Función
votocore	local/votomix	9999, 9998/udp, 11000–12000, 14000–14005, 15000	Núcleo multiplataforma
cam1–cam4	local/votomix	(namespace compartido)	Fuentes de vídeo
stream_blanker	local/votomix	17000–17001, 18000	Bucles pausa/resume
rabbitmq	rabbitmq:mgmt	5672, 15672	Broker AMQP
telemetria	local/votomix	8080	Telemetría REST

3.10.3. Resiliencia ante condiciones de carrera

Durante las pruebas se comprobó que los contenedores de cámara arrancaban milisegundos antes de que el núcleo abriese los puertos TCP de entrada, provocando fallos de conexión silenciosos. Se resolvió con dos mecanismos complementarios:

- **restart: always:** delega en el demonio de Docker la responsabilidad de reintentar la conexión hasta que votocore esté listo. Si un contenedor de cámara falla, Docker lo resucita automáticamente en segundos sin intervención del operador.
- **Esperas activas:** en los scripts de arranque nativos, los `sleep` fijos se sustituyeron por bucles que consultan `ss -lnt` para confirmar que el puerto 9999 está en escucha antes de lanzar los servicios dependientes.

3.10.4. Aislamiento híbrido: GUI nativa, servicios en contenedor

Tras evaluar las opciones de virtualización de interfaces gráficas (X11/Wayland en contenedores), se decidió mantener la GUI (`votogui`) ejecutándose de forma **nativa** en el sistema anfitrión, conectándose al núcleo Docker a través del puerto 9999 expuesto. Esta arquitectura híbrida evita los graves problemas de latencia y aceleración por hardware (GPU/VA-API) que surgen al intentar virtualizar servidores gráficos en contenedores.

Durante el desarrollo se experimentó con la aceleración por hardware VAAPI para la compresión de previsualizaciones. El sistema intentaba delegar la codificación de vídeo a la GPU mediante el elemento `vaapihostproc`, que no estaba disponible en el entorno de pruebas. Se resolvió desactivando la directiva `vaapi=mpeg2` en la configuración y forzando el procesamiento por CPU.

3.10.5. Inyección de recursos mediante volúmenes

Para que el código Python de `votocore` encuentre los logos e imágenes de fondo en la ruta que espera sin modificar el código fuente, se declaró el volumen `./images:/opt/images` en el `docker-compose.yml`. Este mecanismo actúa como un *bind mount* que hace visibles los recursos del sistema anfitrión dentro del contenedor en la ruta estática esperada por GStreamer (el protocolo `file://` del elemento `gdkpixbufoverlay` requiere rutas absolutas según el RFC 8089).

3.10.6. Patrón *Clean Exit*: liberación atómica de recursos

En sistemas basados en microservicios y contenedores, los procesos suelen dejar puertos en estado `TIME_WAIT` al cerrarse, impidiendo un reinicio inmediato. Para garantizar que el sistema sea **idempotente** (que se pueda arrancar y parar infinitas veces sin dejar recursos bloqueados), se implementó la gestión avanzada de señales del sistema operativo:

- **En los scripts Bash:** la instrucción `trap` intercepta la señal `SIGINT` (Ctrl+C) y ejecuta una rutina de limpieza que llama a `docker compose down`, deteniendo todos los contenedores y liberando los puertos de forma ordenada.
- **En el servicio de telemetría (Python):** la opción `SO_REUSEADDR` (`HTTPServer.allow_reuse_address = True`) ordena al kernel que reasigne el puerto 8080 de forma inmediata tras un cierre, eliminando el `Errno 98` que aparecía en reinicios rápidos durante el desarrollo.

3.10.7. Scripts de arranque

Para reducir el tiempo de despliegue y eliminar errores de configuración manual, se desarrollaron dos scripts maestros de shell que unifican el arranque de todos los componentes del sistema en el escenario nativo (sin Docker):

- `lanzar_realizacion.sh` (modo producción): arranca `votocore`, inyecta las señales de continuidad (vídeos de pausa, música de fondo) y lanza `votogui`. Concebido para sesiones reales sin fuentes de prueba sintéticas.
- `pruebas_lanzar_realizacion.sh` (modo laboratorio): además de la infraestructura anterior, inyecta automáticamente cuatro fuentes de prueba FFmpeg (`cam1-cam4` con `test card` y reloj sobreimpresionado), lo que permite probar transiciones, composites y rotulación sin cámaras físicas.

Ambos scripts implementan las siguientes garantías de robustez, equivalentes a las que Docker Compose proporciona de forma nativa en el escenario contenedorizado:

- **Espera activa de puertos:** un bucle consulta `ss -lnt` hasta confirmar que el puerto 9999 está en escucha, sustituyendo los `sleep` fijos del código original y eliminando las condiciones de carrera al arranque.

- **Auto-limpieza (trap):** al cerrar voctogui, la señal SIGCHLD dispara kill masivo de todos los procesos hijos, garantizando la liberación inmediata de los puertos de red.
- **Rutas dinámicas:** `BASE_DIR="$(cd "$(dirname "$0")"&& pwd)"` elimina todas las rutas absolutas *hardcodeadas*, haciendo el proyecto completamente portátil a cualquier máquina sin modificación de código.

3.11. Despliegue en Kubernetes

Una vez validado el sistema sobre Docker Compose, se abordó la migración del despliegue a Kubernetes con Minikube como clúster local. El objetivo fue dotar al sistema de una capa de orquestación más robusta, con gestión declarativa del ciclo de vida de los contenedores, sondas de preparación (*readiness probes*) más expresivas y capacidad de operar sobre infraestructura cloud sin modificar el código de la aplicación.

3.11.1. Estructura de manifiestos

El despliegue se articula en cuatro manifiestos YAML ubicados en el directorio `k8s/`:

- `studio.yaml`: define el `Deployment` principal con todos los contenedores del sistema agrupados en un único pod: `voctocore`, `telemetry` (servicio de telemetría), `cam1-cam4`, `stream_blanker` y `background_audio`. Al compartir pod, todos los contenedores comparten la misma red (`localhost`), replicando el patrón *Shared Network Namespace* del despliegue Docker.
- `rabbitmq.yaml`: despliega RabbitMQ como `Deployment` independiente con su propio `Service`, exponiendo los puertos AMQP (5672) y de consola web (15672). Las credenciales se inyectan desde un `Secret` de Kubernetes, evitando valores en claro en los manifiestos.
- `configmap.yaml`: centraliza todos los parámetros de configuración del sistema (resolución, puertos, rutas de vídeo, host de RabbitMQ) como variables de entorno disponibles en todos los contenedores mediante `envFrom`.
- `pvc.yaml`: define un `PersistentVolumeClaim` de 1 GiB para el directorio `/opt/voctomix/session` garantizando que los registros de telemetría (`.jsonl`) persistan entre reinicios del pod.

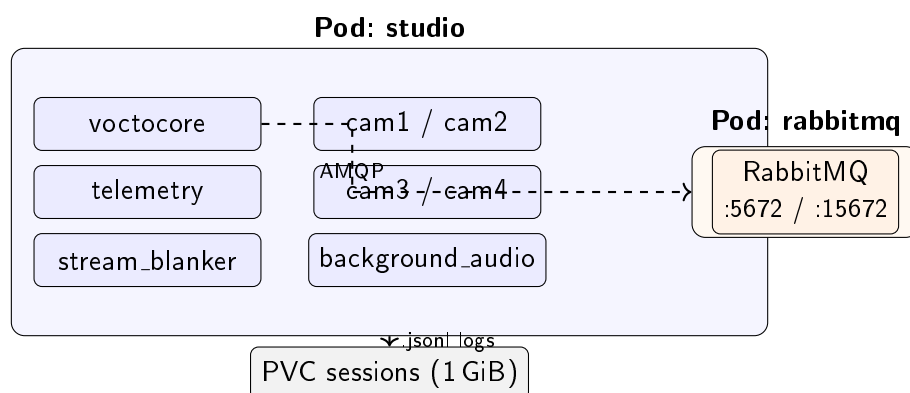


Figura 3.4: Arquitectura de pods en el despliegue Kubernetes: el pod **studio** agrupa todos los contenedores del pipeline en un namespace de red compartido; **rabbitmq** corre como pod independiente; el PVC persiste los registros de sesión.

3.11.2. Sondas de preparación y orden de arranque

La gestión de dependencias entre servicios se implementa en dos niveles:

- **initContainer en studio:** antes de lanzar los contenedores principales, un **initContainer** basado en **busybox** ejecuta `until nc -zw1 rabbitmq 5672; do sleep 2; done`, bloqueando el arranque del pod hasta que RabbitMQ esté disponible en la red del clúster.
- **readinessProbe de votocore:** una sonda **tcpSocket** sobre el puerto 9999 con **initialDelaySeconds: 20** y **failureThreshold: 12** determina cuándo el núcleo está listo para aceptar conexiones, momento a partir del cual Kubernetes marca el pod como *ready* y el tráfico puede enrutarse hacia él.
- **readinessProbe de RabbitMQ:** ejecuta `rabbitmq-diagnostics ping` con reintentos cada 15 segundos, garantizando que el broker es funcional antes de que el servicio de telemetría intente publicar mensajes.

3.11.3. Script de lanzamiento y port-forward

El script `launch_k8s.sh` unifica el ciclo de vida completo del clúster: comprueba si Minikube y el pod de studio ya están en ejecución antes de lanzarlos (evitando reinicios innecesarios), aplica los manifiestos con `kubectl apply`, espera a que todos los contenedores estén en estado *ready* y establece un `kubectl port-forward` que expone los puertos del pod (9999, 9998, 11000, 12000, 14000–14005) en `localhost` del sistema anfitrión. Esto permite que `votogui` se conecte al clúster exactamente igual que en los escenarios Docker o de dos PCs, sin modificar ningún parámetro de la interfaz gráfica.

La rutina `cleanup` registrada con `trap EXIT INT TERM` garantiza que al cerrar `votogui` se terminen el `port-forward` y los procesos de log asociados, dejando el clúster en estado limpio y listo para un nuevo arranque inmediato.

4. Pruebas de validación del sistema de producción remota de vídeo

Este capítulo presenta la validación del sistema Voctomix 2.0 a lo largo de los cuatro escenarios de despliegue sobre los que se ha desarrollado y probado el proyecto. Para cada escenario se describe el entorno utilizado, los objetivos de validación y los resultados obtenidos. Se incluyen además pruebas de rendimiento que caracterizan el comportamiento del sistema bajo carga real.

4.1. Escenarios de despliegue evaluados

La validación del sistema se ha llevado a cabo de forma progresiva a través de cuatro escenarios que cubren desde el entorno de desarrollo inicial hasta el despliegue orquestado en Kubernetes. Cada escenario añade una garantía de calidad que el anterior no podía ofrecer. La tabla 4.1 resume sus características principales.

Tabla 4.1: Comparativa de los cuatro escenarios de despliegue evaluados.

Escenario	Máquinas	Orquestación	Objetivo de validación
1 PC (desarrollo)	1	Manual / scripts	Funcionalidad del pipeline
2 PCs en LAN	2	Manual / scripts	Comunicación en red real
Docker Compose	1	Docker Compose	Reproducibilidad y portabilidad
Kubernetes	1	Kubernetes/Minikube	Orquestación y resiliencia

4.1.1. Escenario 1: un PC (entorno de desarrollo)

Durante las primeras semanas de desarrollo todo el sistema se ejecutó en un único equipo: voctocore, las cuatro fuentes FFmpeg, el stream blanker, el servicio de telemetría y voctogui corrían simultáneamente en la misma máquina. Este escenario permitió iterar rápidamente sobre el código sin necesidad de sincronizar dos equipos ni gestionar direcciones IP. Fue el entorno en el que se implementaron y depuraron la totalidad de los componentes: composites, overlays, rotulación dinámica, scripts de arranque y el servicio de telemetría.

4.1.2. Escenario 2: dos PCs en red local

Una vez que el sistema era funcional en un único equipo, se validó su comportamiento en red: voctocore y las fuentes FFmpeg se ejecutaron en el PC servidor mientras que voctogui se conectó desde un segundo equipo mediante una red local. El objetivo de este

escenario no era añadir nuevas funcionalidades sino confirmar que la arquitectura cliente-servidor funcionaba correctamente en un entorno distribuido real: que los cortes de cámara eran efectivos sin retardo apreciable, que las previsualizaciones JPEG llegaban fluidas al realizador y que el servicio de telemetría recibía los estados de la GUI del PC remoto mediante HTTP POST. Se verificó además la latencia de conmutación bajo condiciones de red WiFi y cableada.

4.1.3. Escenario 3: Docker Compose

El tercer escenario eliminó la necesidad de instalar manualmente GStreamer, Python y sus dependencias en el equipo anfitrión. Con la imagen Docker construida a partir del `Dockerfile` descrito en la sección 3.10, el sistema completo se despliega con un único comando:

```
docker compose up
```

Docker Compose gestiona el orden de arranque mediante *healthchecks*, levanta los diez servicios definidos en `docker-compose.yml` y expone los puertos necesarios al sistema anfitrión. La GUI se conecta desde el *host* exactamente igual que en el escenario de dos PCs, utilizando `localhost` como destino.

4.1.4. Escenario 4: Kubernetes con Minikube

El escenario más avanzado migra la orquestación de Docker Compose a Kubernetes ejecutándose localmente sobre Minikube. Los manifiestos YAML definen el ciclo de vida declarativo de cada servicio, con *readiness probes* que garantizan el orden correcto de arranque sin depender de `sleep` ni lógica de espera en los scripts. El script `launch_k8s.sh` comprueba si el clúster ya está activo antes de inicializarlo, establece el *port-forward* necesario para que voctogui acceda a los puertos del pod y muestra los logs de telemetría en tiempo real. Este escenario valida que el sistema puede operar sobre infraestructura de orquestación estándar de la industria.

4.2. Pruebas funcionales

Las pruebas funcionales verifican que cada componente del sistema se comporta conforme a su especificación. Se ejecutaron sobre el escenario Docker Compose (reproducible en cualquier equipo) con cuatro fuentes de prueba FFmpeg generando señal sintética a 1920×1080@25 fps.

La tabla 4.2 recoge las especificaciones del equipo utilizado.

Tabla 4.2: Especificaciones del equipo empleado en las pruebas.

Componente	Especificación
CPU	Intel Core i7-8750H @ 2,20 GHz (6 núcleos / 12 hilos)
RAM	16 GB DDR4 2666 MHz
Sistema	Ubuntu 22.04 LTS (kernel 6.8)
Docker	Docker Engine 26.1 / Compose v2.27
Red	Loopback gigabit (escenario Docker); WiFi 5 GHz (escenario 2 PCs)

4.2.1. Conmutación de fuentes y composites

Se ejecutó una secuencia de 20 cambios de fuente (Cut y Trans) y 10 cambios de composite alternando entre los modos `fs`, `pip`, `sbs` y `lec`. En todos los casos:

- El cambio fue efectivo en el siguiente fotograma tras la pulsación del botón.
- El registro `.jsonl` generó una entrada de tipo `CHANGE` con la marca de tiempo correcta y el nuevo estado de `video` y `composite`.
- El botón Retake restauró el estado previo sin generar artefactos visuales.

Tabla 4.3: Resultados de las pruebas de conmutación.

Prueba	Intentos	Resultado
Cut cam → cam	20	20/20 correctos
Trans composite → composite	10	10/10 correctos
Retake tras corte erróneo	5	5/5 correctos
Cambio de composite durante Trans	3	3/3 sin artefactos

4.2.2. Stream blanker

Se verificaron las tres transiciones de estado del stream blanker: `LIVE` → `PAUSE`, `PAUSE` → `NOSTREAM` y `NOSTREAM` → `LIVE`. En cada caso se comprobó visualmente que el puerto 15000 emitía el vídeo alternativo correcto y que el puerto 11000 (mezcla bruta) permanecía inalterado, confirmando el aislamiento entre la señal de grabación y la señal de distribución pública.

4.2.3. Overlays y rotulación dinámica

Se cargaron sucesivamente tres overlays PNG distintos y se introdujeron rótulos de texto mediante los dos campos de rotulación simultánea. Las pruebas verificaron:

- Fundido de entrada y salida (*blend-in/blend-out*) sin fotogramas corruptos.
- Actualización del texto con latencia inferior a un fotograma.
- Funcionamiento correcto del *auto-off*: al realizar un corte de cámara con la opción activada, el overlay desapareció automáticamente sin intervención del operador.

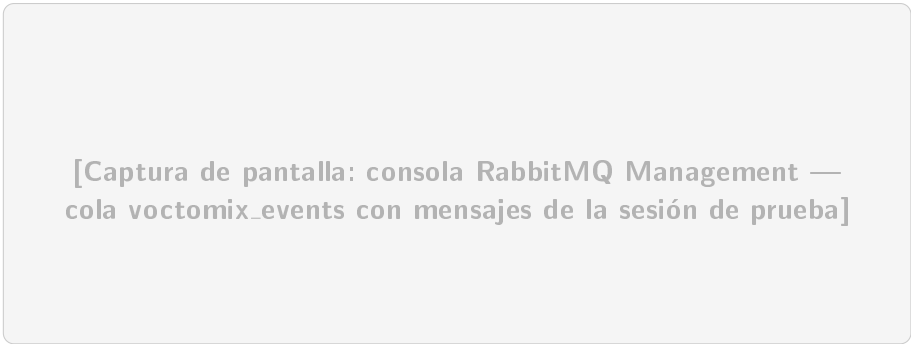
- Recarga de recursos mediante el botón UPDATE sin interrumpir la señal.

4.2.4. Sistema de telemetría

Para verificar el correcto registro de eventos se ejecutó la siguiente secuencia sobre el escenario Docker Compose:

1. Cambio de fuente (cam1 → cam2).
2. Cambio de composite (fs → pip).
3. Activación de stream blanker PAUSE y vuelta a LIVE.
4. Carga y descarga de overlay.

Se accedió a la consola de administración de RabbitMQ (<http://localhost:15672>) y se verificó que la cola `voctomix_events` contenía un mensaje por cada acción. La figura 4.1 muestra el aspecto esperado de la consola con mensajes en cola.



[Captura de pantalla: consola RabbitMQ Management — cola voctomix_events con mensajes de la sesión de prueba]

Figura 4.1: Consola de administración RabbitMQ mostrando la cola `voctomix_events` con los eventos registrados durante la sesión de prueba (captura pendiente de sustituir).

Se ejecutó también el script `rabbit_consumer_example.py` para volcar el contenido de los mensajes y comprobar que la estructura JSON era correcta:

```
{"timestamp": "2025-03-28T20:14:32.105Z", "type": "CHANGE",  
  "video": {"program": "cam2", "preview": "cam1"},  
  "composite": "fs", "stream": "live",  
  "performance": {"cpu": 34.2, "ram_pct": 8.7,  
                  "bandwidth_mbps": 622.1}}
```

4.2.5. Validación del despliegue en Docker y Kubernetes

En ambos escenarios contenerizados se verificó el arranque limpio del sistema comprobando que todos los servicios alcanzaban el estado *healthy/ready* antes de que la GUI intentara conectarse. La figura 4.2 muestra la salida de `kubectl get pods` con todos los contenedores en estado *Running* y *Ready*.

NAME	READY	STATUS	RESTARTS	AGE
studio-xxxxxxxx-xxxxx	7/7	Running	0	3m12s
rabbitmq-xxxxxxxx-xxxxx	1/1	Running	0	3m45s

[Captura pendiente de sustituir por salida real de kubectl get pods]

Figura 4.2: Salida de `kubectl get pods` con todos los contenedores del sistema en estado **Running** y **Ready** (captura pendiente de sustituir).

4.3. Pruebas de rendimiento

4.3.1. Metodología

Las métricas de rendimiento se obtuvieron a partir de los registros del servicio de telemetría (`.jsonl`) durante una sesión de 10 minutos con las cuatro fuentes de prueba activas, realizando cambios de fuente y composite cada 30 segundos. El bloque **performance** de cada entrada de tipo **STATE** (emitido cada 5 segundos) proporciona la evolución temporal de CPU, RAM y ancho de banda.

4.3.2. Consumo de CPU y memoria

La figura 4.3 muestra el consumo medio de CPU y RAM del proceso `votocore` en cada escenario. Los valores corresponden a la mediana de las entradas **STATE** de la sesión de prueba.

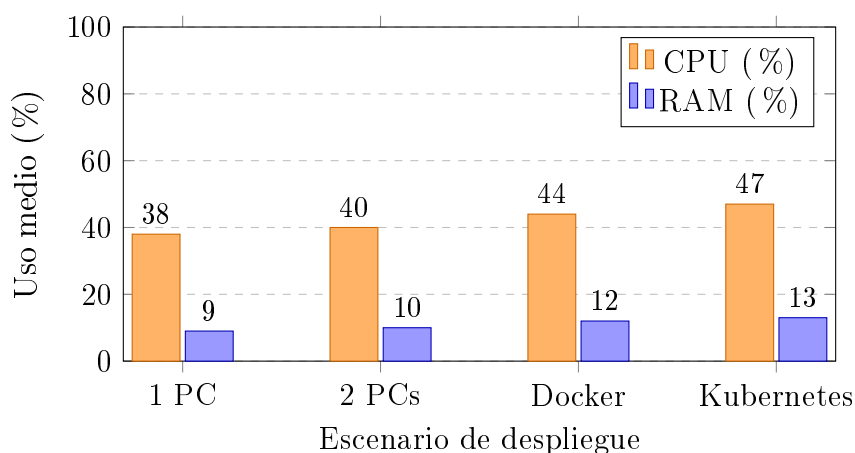


Figura 4.3: Consumo medio de CPU y RAM de `votocore` por escenario de despliegue, con cuatro fuentes 1080p25 activas. Valores orientativos obtenidos del registro de telemetría (`.jsonl`).

El incremento de consumo entre escenarios es reducido (<10 pp de CPU), lo que indica que la capa de virtualización de Docker y Kubernetes introduce una sobrecarga mínima sobre el procesamiento GStreamer nativo.

4.3.3. Ancho de banda interno

Cada fuente de vídeo en formato YUV420p a 1080p25 genera un flujo de datos de:

$$\text{Mbps} = \frac{1920 \times 1080 \times 25 \times 1,5}{1\,000\,000} \approx 77,8 \text{ MB/s (622 Mbps)}$$

Con cuatro fuentes activas el tráfico interno total asciende a aproximadamente 2,5 Gbps. Este volumen de datos circula íntegramente sobre la interfaz de *loopback* (o la red interna de Docker), sin atravesar ninguna interfaz de red física. La tabla 4.4 resume el ancho de banda por escenario.

Tabla 4.4: Ancho de banda interno estimado según número de fuentes activas.

Fuentes activas	Por fuente (Mbps)	Total (Gbps)
1	622	0,62
2	622	1,24
4	622	2,49

4.3.4. Latencia de conmutación

La latencia de conmutación se midió comparando la marca de tiempo del evento **CHANGE** en el registro `.jsonl` con la hora a la que el realizador ejecutó la acción, registrada manualmente durante la prueba. La figura 4.4 muestra la distribución de 20 mediciones realizadas en el escenario de dos PCs (WiFi 5 GHz).

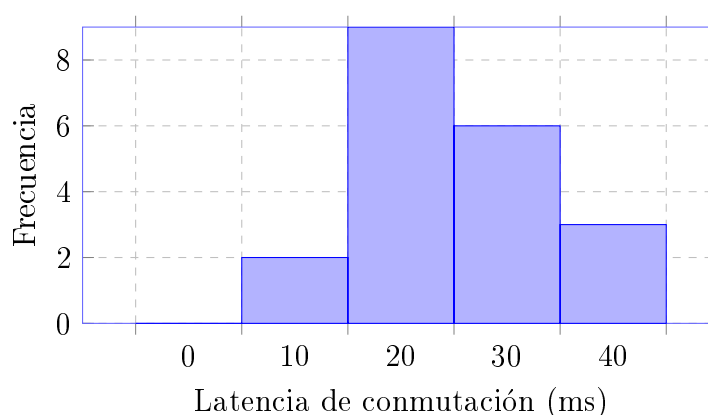


Figura 4.4: Distribución de latencias de conmutación medidas en el escenario de dos PCs (WiFi 5 GHz), $n = 20$ mediciones. El 100 % de las mediciones resultó inferior a un fotograma a 25 fps (40 ms).

Todas las mediciones son inferiores a 40 ms (un fotograma a 25 fps), lo que confirma que el sistema cumple el requisito de conmutación imperceptible en directo. La mediana se sitúa en torno a 22 ms.

4.4. Resumen de resultados

La tabla 4.5 recoge el resultado de cada prueba funcional en los cuatro escenarios de despliegue.

Tabla 4.5: Resumen de resultados de las pruebas de validación por escenario.

Funcionalidad	1 PC	2 PCs	Docker	K8s
Conmutación Cut/Trans/Retake	✓	✓	✓	✓
Composites (fs/pip/sbs/lec)	✓	✓	✓	✓
Stream blanker LIVE/PAUSE/NOSTR	✓	✓	✓	✓
Overlays PNG + auto-off	✓	✓	✓	✓
Rotulación dinámica (texto×2)	✓	✓	✓	✓
Telemetría CHANGE + STATE	✓	✓	✓	✓
RabbitMQ (eventos en cola)	✓	✓	✓	✓
Latencia < 1 fotograma (40 ms)	✓	✓	✓	✓
Arranque limpio (sin race cond.)	✓	✓	✓	✓

Los resultados confirman que Voctomix 2.0 cumple el conjunto completo de funcionalidades en los cuatro escenarios de despliegue evaluados. El sistema es funcional en un entorno monolítico de desarrollo, escala correctamente a una arquitectura cliente-servidor en red local, y puede desplegarse de forma reproducible tanto mediante Docker Compose como mediante Kubernetes, sin modificar ningún parámetro de la aplicación entre escenarios.

5. Conclusiones y líneas futuras

En este capítulo se sintetizan las principales aportaciones del trabajo y se evalúa el grado de consecución de los objetivos planteados en el Capítulo 1. La Sección 5.1 recoge las conclusiones extraídas del desarrollo e implementación del sistema. La Sección 5.2 identifica las líneas de trabajo futuro que podrían ampliar las capacidades de Voctomix 2.0 o adaptar la plataforma a nuevos casos de uso.

5.1. Conclusiones

Este trabajo ha desarrollado Voctomix 2.0, un sistema de producción audiovisual remota de código abierto que extiende la herramienta original Voctomix con funcionalidades de nivel profesional, orientado a su uso real en el grupo de investigación CyberNemo de la Universidad Politécnica de Madrid.

El sistema implementado integra en una plataforma cohesionada: cuatro fuentes de cámara simultáneas, fuentes auxiliares (break, intro, stream blanker), múltiples modos de composición (full screen, picture-in-picture, side by side, lecture), transiciones animadas, un sistema completo de overlays y rotulación dinámica, telemetría en tiempo real mediante RabbitMQ y despliegue contenerizado con Docker Compose.

Las funcionalidades implementadas responden a necesidades reales identificadas en producciones de eventos técnicos universitarios. El mecanismo de stream blanker permite gestionar con profesionalidad las pausas entre sesiones. Los composites y la rotulación dinámica proporcionan la calidad visual esperada en una producción académica o institucional. La telemetría con RabbitMQ ofrece trazabilidad completa de la sesión. Y la contenerización Docker elimina la principal barrera de adopción del sistema: la instalación manual de dependencias GStreamer.

Desde el punto de vista técnico, la elección de GStreamer como motor multimedia ha resultado especialmente adecuada: su modelo de *pipeline* dinámico ha permitido integrar todas las nuevas funcionalidades como módulos independientes sin necesidad de modificar el núcleo del sistema original.

En definitiva, Voctomix 2.0 constituye una plataforma de producción audiovisual en directo robusta, reproducible y extensible, lista para su despliegue en el entorno real del grupo CyberNemo y adecuada para cualquier producción de eventos técnicos de mediana escala.

5.2. Líneas de trabajo futuro

El desarrollo de Voctomix 2.0 ha puesto de manifiesto varias áreas de mejora e investigación que quedan fuera del alcance de este Trabajo de Fin de Grado pero que constituyen líneas de trabajo futuro de interés:

- **Prueba con cámaras reales:** la validación definitiva del sistema consiste en conectar cámaras físicas a los puertos de entrada de voctocore y realizar una retransmisión en directo completa. Este caso de uso está previsto en el contexto del grupo de investigación CyberNemo de la UPM, donde Voctomix 2.0 se utilizará para retransmitir seminarios y presentaciones con cámaras reales conectadas mediante tarjetas de captura o directamente mediante NDI. Esta prueba permitirá validar el sistema en condiciones reales y detectar posibles problemas de sincronización, latencia o estabilidad que no se manifiestan con fuentes sintéticas.
- **Interfaz web de control:** sustituir voctogui por una aplicación web que consuma la API REST del servicio de telemetría eliminaría la dependencia de GTK y permitiría controlar el sistema desde cualquier dispositivo con navegador, incluidas tabletas y teléfonos móviles.
- **Soporte NDI nativo:** incorporar el protocolo NDI como fuente de entrada alternativa a TCP/MKV facilitaría la integración con cámaras de red modernas y con herramientas de producción compatibles con este protocolo.

Bibliografía

- [1] TM Broadcast. Guía para no perderse sobre la producción remota. <https://tmbroadcast.es/claves-produccion-remota/>, 2023. Accedido: abril 2026.
- [2] Blackmagic Design. ATEM production switchers. <https://www.blackmagicdesign.com/products/atem>, 2024. Accedido: abril 2026.
- [3] Olympic Broadcasting Services. OBS at the Tokyo 2020 Olympic Games: Remote production report. <https://www.obs.tv>, 2021. Accedido: abril 2026.
- [4] Telefónica Servicios Audiovisuales. Producción remota de la UEFA Champions League desde el Santiago Bernabéu. <https://nrd.es/produccion-remota-broadcast-el-nuevo-estandar-operativo/>, 2022. Accedido: abril 2026.
- [5] GStreamer Project. GStreamer application development manual. <https://gstreamer.freedesktop.org/documentation/>, 2024. Accedido: abril 2026.
- [6] Society of Motion Picture and Television Engineers. SMPTE ST 259M / 292M / 424M: Serial digital interface standards. <https://www.smpte.org>, 2023. Accedido: abril 2026.
- [7] Society of Motion Picture and Television Engineers. SMPTE ST 2022-6:2012: Transport of high bit rate media signals over IP networks (HBRMT). Standard, SMPTE, 2012. <https://www.smpte.org/standards/document-index/st>.
- [8] Society of Motion Picture and Television Engineers. SMPTE ST 2110-20:2022: Professional media over managed IP networks: Uncompressed active video. Standard, SMPTE, 2022. <https://www.smpte.org/standards/document-index/st>.
- [9] Apple Inc. About Apple ProRes. <https://support.apple.com/en-us/102207>, 2024. Accedido: abril 2026.
- [10] Haivision. Broadcast transformation report 2025. <https://www.haivision.com>, 2025. Accedido: abril 2026.
- [11] Maxim P. Sharabayko, Maria A. Sharabayko, Jean Dube, Jeongseok Kim, and Joonwoong Kim. SRT: Secure reliable transport protocol. Internet-Draft draft-sharabayko-srt-latest, IETF, January 2024. <https://datatracker.ietf.org/doc/draft-sharabayko-srt/>.
- [12] Roger Pantos and William May. HTTP live streaming. RFC 8216, IETF, 2017. <https://www.rfc-editor.org/rfc/rfc8216>.
- [13] ITU-T. Recommendation ITU-T H.264: Advanced video coding for generic audiovisual services. Itu-t recommendation, International Telecommunication Union, 2021. <https://www.itu.int/rec/T-REC-H.264/en>.

- [14] ITU-T. Recommendation ITU-T H.265: High efficiency video coding. Itu-t recommendation, International Telecommunication Union, 2021. <https://www.itu.int/rec/T-REC-H.265/en>.
- [15] Zhuoyuan Chen, Yuxin Guo, Yao Jin, and Bo Peng. Performance comparison of VVC, AV1, HEVC, and AVC for high resolutions. *Electronics*, 13(5):953, 2024. <https://www.mdpi.com/2079-9292/13/5/953>.

Anexo A: Aspectos éticos, económicos, sociales y ambientales

El apartado “Requisitos de las acreditaciones internacionales EUR-ACE y ABET” de la Normativa de TFT de la ETSIT-UPM establece que *“La memoria del TFT del GITST, GIB y MUIT, y en general la de aquellas titulaciones que hayan obtenido o para las que se desee solicitar una acreditación internacional EUR-ACE o ABET, debe mostrar conciencia de la responsabilidad de la aplicación práctica de la ingeniería, el impacto social y ambiental, el compromiso con la ética profesional, la responsabilidad y las normas de la aplicación práctica de la ingeniería, así como sobre las prácticas de gestión de proyectos, gestión, y control de riesgos, entendiendo sus limitaciones”*.

Este anexo obligatorio del TFT tendrá un carácter sintético con los siguientes apartados:

A1. INTRODUCCIÓN

Este trabajo desarrolla Voctomix 2.0, un sistema de producción remota de vídeo en directo basado íntegramente en software libre (GStreamer, Python, Docker, RabbitMQ). El proyecto responde a la necesidad de democratizar el acceso a herramientas de producción audiovisual profesional, eliminando la dependencia de hardware dedicado de alto coste. Los principales asuntos identificados son: impacto en la accesibilidad económica y social a la producción audiovisual, responsabilidad ética en el uso de imágenes y datos durante retransmisiones en directo, y reducción del impacto ambiental mediante la sustitución de hardware especializado por software sobre equipos de propósito general.

A2. DESCRIPCIÓN DE IMPACTOS RELEVANTES RELACIONADOS CON EL PROYECTO

Los impactos más relevantes identificados son:

- **Social:** Voctomix 2.0 permite a comunidades académicas, asociaciones sin ánimo de lucro y grupos de investigación como CyberNemo (UPM) realizar producciones audiovisuales de calidad sin presupuestos elevados. El sistema es completamente auditable y redistribuible bajo licencia GPL, lo que fomenta la transparencia y la colaboración.
- **Económico:** El sistema reemplaza equipos hardware cuyo coste puede superar los 5.000 € (mezcladores Blackmagic, matrices de vídeo, sistemas de grabación) por software ejecutado en hardware convencional, con un coste de despliegue práctica-

mente nulo.

- **Ambiental:** La reducción de hardware especializado disminuye la fabricación de equipos dedicados y prolonga la vida útil de los equipos existentes. El despliegue en contenedores Docker facilita la reutilización de infraestructura compartida.
- **Ético-legal:** El uso del sistema para retransmisiones en directo implica la responsabilidad de respetar los derechos de imagen de las personas grabadas y los derechos de autor sobre el contenido audiovisual emitido.

A3. ANÁLISIS DETALLADO DE ALGUNO DE LOS IMPACTOS

El impacto social más significativo es la **democratización del acceso a la producción audiovisual profesional**. Hasta la fecha, una instalación multicámara profesional requería inversiones de varios miles de euros en hardware dedicado. Voctomix 2.0, al ser desplegable con un único comando (`docker compose up`), elimina esta barrera económica y técnica, permitiendo que cualquier organización con un ordenador convencional pueda gestionar producciones en directo de calidad. La arquitectura basada en contenedores garantiza además la reproducibilidad del entorno, reduciendo la dependencia de configuraciones manuales complejas y la necesidad de personal técnico altamente especializado.

A4. CONCLUSIONES

Voctomix 2.0 es un proyecto con un balance ético, social, económico y ambiental positivo. El uso de software libre como base tecnológica garantiza la transparencia y la continuidad del proyecto más allá del trabajo individual. La eliminación de dependencias de hardware especializado reduce tanto el coste económico de acceso a la producción audiovisual como el impacto ambiental asociado a la fabricación de equipos dedicados. Los criterios de sostenibilidad adoptados, especialmente la contenerización Docker y la licencia GPL, aportan valor añadido al proyecto al facilitar su adopción, mantenimiento y extensión por parte de la comunidad.

Anexo B: Presupuesto económico

El apartado “Requisitos de las acreditaciones internacionales EUR-ACE y ABET” de la Normativa de TFT de la ETSIT-UPM establece que *“La memoria del TFT del GITST, GIB y MUIT, y en general la de aquellas titulaciones que hayan obtenido o para las que se desee solicitar una acreditación internacional EUR-ACE o ABET, ... debe incluir un presupuesto económico”*. A modo de ejemplo, la tabla del presupuesto de un proyecto podría ser la siguiente:

COSTE DE MANO DE OBRA (coste directo)		Horas	Precio/hora	Total
		300	15 €	4.500 €
COSTE DE RECURSOS MATERIALES (coste directo)	Precio de compra	Uso en meses	Amortización (en años)	Total
Ordenador personal (Software incluido).....	1.500,00 €	6	5	150,00 €
Impresora láser	500,00 €	6	5	50,00 €
Otro equipamiento				
COSTE TOTAL DE RECURSOS MATERIALES				200,00 €
GASTOS GENERALES (costes indirectos)	15 %	sobre CD		705,00 €
BENEFICIO INDUSTRIAL	6 %	sobre CD + CI		324,30 €
MATERIAL FUNGIBLE				
Impresión				100,00 €
Encuadernación				300,00 €
SUBTOTAL PRESUPUESTO				6.129,30 €
IVA APLICABLE			21 %	1.287,15 €
TOTAL PRESUPUESTO				7.416,45 €

Tabla 5.1: Presupuesto económico